Planning & Reinforcement Learning

Artificial Intelligence

Jacopo Parretti

II Semester 2025-2026

Contents

Part I

Introduction to Reinforcement Learning

1 What is Reinforcement Learning

1.1 From Traditional Computer Science to Machine Learning

In **traditional computer science**, computers are explicitly programmed for every task they perform — a program is a function mapping inputs to outputs via instructions written by a human.

The **machine learning paradigm** inverts this: **examples** are provided to machines, and machines learn models capable of performing tasks based on those examples. Both a hand-written program and a learned model are functions; the fundamental difference lies in how the function is obtained.

1.2 The Nature of Learning

A basic observation is that **we learn by interacting with our environment**. Infants have no explicit teacher but develop knowledge through a direct sensorimotor connection to their surroundings. Exercising this connection produces information about:

- **cause-effect relationships** (consequences of actions)
- what to do to achieve **goals**

1.3 Learning from Interaction

We are aware of how our environment responds to what we do, and we seek to influence what happens through our behavior. **Learning from interaction** is the foundational idea underlying nearly all theories of learning and intelligence.

1.4 Reinforcement Learning: Definition and Features

Reinforcement Learning

Reinforcement Learning (RL) is a computational approach to **learning from interaction**. The agent learns to **map situations to actions** so as to **maximize** a numerical **reward signal**.

Two most important features distinguish RL from other learning paradigms:

1. **Trial-and-error search**: the learner is *not* told which action to take in each situation (as in supervised learning). It must **discover** which actions yield the most reward by trying them.
2. **Delayed reward**: actions may affect not only the immediate reward but also the next state and, through that, all **subsequent rewards**.

1.5 RL as Problem, Solution Methods, and Research Field

RL is simultaneously:

- a **problem**

- a **class of solution methods**
- a **research field** that studies this problem and its solution methods

It is important to distinguish the three to avoid confusion. The **problem** of RL draws ideas from dynamical systems theory and optimal control of incompletely-known Markov Decision Processes.

1.6 Main Ingredients

The RL problem involves five main elements:

- **Agent**: the learner and decision maker
- **Environment**: everything the agent interacts with
- **State**: a representation of the current situation
- **Action**: what the agent can do at each step
- **Reward**: scalar feedback from the environment

The agent selects an action, the environment transitions to a new state and emits a reward; the loop repeats. All these elements are formalized by **Markov Decision Processes (MDPs)**, introduced in subsequent lectures.

1.7 RL vs Supervised Learning

Supervised learning learns from a training set of **labeled samples** provided by an external supervisor. Each sample encodes a mapping

$$\text{situation} \rightarrow \text{correct action (label)}$$

The goal is to generalize to situations not seen during training.

Supervised learning is **not adequate** for learning from interaction because:

- In interactive problems it is impractical to obtain informative labeled training sets
- The agent must learn from its own experience, not from pre-labeled data

1.8 RL vs Unsupervised Learning

Unsupervised learning seeks hidden **structure** in collections of unlabeled data. Reinforcement learning, instead, tries to **maximize a reward signal** rather than discover structure.

Discovering structure in an agent's experience may be useful, but it does not in itself address the problem of maximizing reward.

Reinforcement learning is a **third machine learning paradigm**, alongside supervised and unsupervised learning.

1.9 The Exploration-Exploitation Trade-off

A key challenge in RL, absent from other forms of learning, is the optimization of the **exploration-exploitation trade-off**:

- **Exploration**: select actions *never tried* in a given situation, to learn what happens.
  - Risky in terms of immediate reward acquisition

  - Informative about environment dynamics
- **Exploitation**: select actions *already tried* and found effective in producing reward.
  - Safe in terms of reward acquisition
  - Not informative

The agent must try a variety of actions and progressively favor those that appear best. On **stochastic tasks** each action must be tried many times to get a reliable estimate of its expected reward. The exploration-exploitation dilemma is still **unsolved** in the general case.

1.10 RL Agents as Components of Larger Systems

An RL agent can be a **component of a larger system** (e.g., an agent monitoring the charge level of a robot's battery). In this case the agent's environment is the rest of the system together with the external world.

1.11 Interaction with Other Disciplines

RL has substantive interactions with: Artificial Intelligence, Machine Learning, Statistics, Optimization, Operations Research, Control Theory, Psychology, and Neuroscience.

2 Examples and Applications of RL

RL has been applied successfully across a wide range of domains:

- **Game playing**: IBM's Deep Blue vs Kasparov in chess (1997); **AlphaGo** reaching superhuman performance in Go (DeepMind, 2016); Atari, Backgammon, Blackjack, Tic-tac-toe.
- **Autonomous vehicles**: learning to drive a car, helicopter control (UAV), quadcopter control.
- **Robot planning and control**: robot navigation and manipulation in industrial environments.
- **Industrial control**: adaptive controller adjusting parameters of a petroleum refinery's operation in real time (cyber-physical systems).
- **Mobile robotics**: a robot deciding whether to enter a new room in search of trash or to return to a battery recharging station.
- **Smart buildings**: autonomous agent controlling air quality and thermal comfort.
- **Operations research**: pricing, vehicle routing.
- **Spoken dialog systems**: e.g., chatbots.
- **Data center energy optimization**.
- **Self-managing network systems**.
- **Computational finance**.

2.1 Common Features Across Examples

All the above share a set of structural properties:

- **Interaction** between agent and environment
- The agent has a **goal**

- **Uncertainty** about the environment (effects of actions cannot be fully predicted)
- Actions affect the **future** state of the environment
- Presence of indirect and **delayed consequences** of actions
- The agent can use its experience to improve its performance over time → **Adaptation**

3 Elements of Reinforcement Learning

An RL system is characterized by four main elements.

3.1 Policy

Definition 3.1 (Policy)**.** A **policy** π defines the agent's way of behaving at a given time and in a given situation:

$$\pi : \text{state} \ \rightarrow \ \text{action}$$

A policy may be implemented as a function, a lookup table, or a search process. It may be **deterministic** or **stochastic**. Generating the policy is the **target of reinforcement learning**.

3.2 Reward Signal

Definition 3.2 (Reward Signal)**.** The **reward signal** defines the goal of the RL problem. At each step, the environment sends the agent a scalar $R_t \in \mathbb{R}$ (immediate pleasure/pain):

$$r : \text{state, action} \ \rightarrow \ \text{reward}$$

The agent's objective is to **maximize the total reward over long runs**. Reward signals may be deterministic or stochastic functions of state and action.

3.3 Value Function

Definition 3.3 (Value Function)**.** The **value function** specifies what is good in the **long run** (while the reward captures what is good *immediately*).

- **State value function**: $V(s)$ is the total amount of reward an agent can expect to accumulate over the future, starting from state s.

$$\text{State Value Function:} \quad s \ \rightarrow \ V(s)$$

- **State-action value function**: $Q(s, a)$ is the total expected reward starting from s and taking action a.

$$\text{State-Action Value Function:} \quad (s, a) \ \rightarrow \ Q(s, a)$$

It is much harder to determine values than rewards. Hence, **methods for efficiently estimating values are key elements of RL algorithms**.

3.4 Model of the Environment

Definition 3.4 (Environment Model)**.** A **model of the environment** is a mathematical model that mimics the behavior of the environment and allows to infer it:

$$\text{model} : \text{state, action} \ \rightarrow \ \text{next state / reward}$$

Models are used for **planning**, i.e., choosing immediate actions by considering possible future situations.

- **Model-based RL**: uses an explicit model of the environment to select actions.
- **Model-free RL**: does not use the environment model; it is an explicit trial-and-error learner.

4 An Extended Example: Tic-Tac-Toe

The problem: play Tic-Tac-Toe against an **imperfect opponent**. We want to construct a player that finds the opponent's weaknesses and learns to **maximize its probability of winning**.

4.1 Problems with Classical Techniques

Classical approaches require a **complete specification of the opponent**:

- **Minimax** (game theory): minimizes worst-case loss.
- **Dynamic programming**: computes the optimal solution given full model.

Since the opponent is imperfect and unknown, this information must be estimated from experience. One idea: learn the model of the opponent's behavior from many games, then apply dynamic programming — this is not far from what RL does.

4.2 Evolutionary Methods

Evolutionary methods search the space of all possible policies:

- Each policy in the population is evaluated by playing several games; winning probability is estimated.
- Better policies are selected (**hill-climbing** in policy space).

This method can in principle find the best policy, but it is often **inefficient**: what happens *during* the games is ignored, discarding valuable online information.

4.3 Value Function Approach

A more efficient approach uses a **value function** learned online:

1. Set up a table $V(s)$ for each game state s, initialized as:
   - $V(s) = 1$ if s is a winning state
   - $V(s) = 0$ if s is a losing or drawn state
   - $V(s) = 0.5$ otherwise
2. Play many games. Most of the time, select the move leading to the state with the highest value (**greedy move**). Occasionally take a random **exploratory move**.
3. After each greedy move, **back up** the value of the previous state using the **temporal-difference update rule**: let S_t be the state before the greedy move and S_{t+1} the state after; then
   $$V(S_t) \leftarrow V(S_t) + \alpha[V(S_{t+1}) - V(S_t)]$$
   where $\alpha > 0$ is a small **step-size** (learning rate). This moves $V(S_t)$ a fraction α toward $V(S_{t+1})$.

This is an instance of a **temporal-difference (TD) learning rule**.

Convergence

If α is reduced properly over time, the method converges to the true probabilities of winning from each state under optimal play, yielding an **optimal policy against the specific opponent**. If α is kept positive, the policy can also adapt to a slowly changing opponent. The Tic-Tac-Toe player is **model-free**.

4.4 Evolutionary vs Value Function Methods

Both methods search the policy space, but they differ in how they use information:

- **Evolutionary methods** evaluate a fixed policy over several complete games; information collected *during* games is discarded.
- **Value function methods** evaluate individual states; they take advantage of information available *during the course of play.*

This makes value function methods **more efficient** in most cases.

4.5 General Applicability

RL methods are not limited to two-player games. They apply to:

- Problems with **no external opponent** (game against the environment)
- **Non-episodic** (continuing) tasks
- **Continuous-time** problems
- Problems with very **large or infinite state spaces** (e.g., backgammon with $\approx 10^{20}$ states, using neural network function approximation)
- Problems where the state is **partially observable**
- Settings where **prior knowledge** can be incorporated

Part II

The k-Armed Bandit Problem

5 Evaluative vs Instructive Feedback

A central distinction in learning theory separates two types of training signal:

- **Instructive feedback** (supervised learning): the learner is told the correct action independently of what it actually did. The feedback specifies the right answer.
- **Evaluative feedback** (RL): the feedback only says *how good* the action taken was, without indicating whether it was the best or worst possible. The signal depends entirely on the action chosen.

Evaluative feedback requires **active exploration**: the agent must try actions to learn which are good; no external supervisor reveals the optimal choice. The k-armed bandit is the canonical

setting for studying evaluative feedback in isolation, before adding the full complexity of state transitions.

6 Problem Formulation

6.1 Setup

k-Armed Bandit

At each discrete time step t, the agent selects one of k actions $a \in \mathcal{A} = \{1, \ldots, k\}$. After selection it receives a scalar reward R_t drawn i.i.d. from a stationary distribution that depends only on the action chosen. The objective is to maximize the expected cumulative reward over a time horizon T.

The name originates from a slot machine (*one-armed bandit*) with k levers; pulling a lever yields a random payout.

6.2 Action Values

The **true value** of action a is its expected reward:

$$q_*(a) = \mathbb{E}[R_t \mid A_t = a].$$

The agent does not know $q_*(a)$ and must estimate it. Let $Q_t(a)$ denote the **estimated value** of action a at time t. The goal is to make $Q_t(a) \approx q_*(a)$.

Remark. The bandit problem is **non-associative**: there is only one situation (state). In full RL, actions must be associated with the situation in which they are taken.

7 Exploration vs Exploitation

7.1 The Dilemma

At any time t, the **greedy action** is $\arg\max_a Q_t(a)$ — the action with the highest current estimate. Choosing it is **exploitation**: it maximizes expected immediate reward given current knowledge.

Choosing a non-greedy action is **exploration**: it may yield lower immediate reward but produces information that can improve future decisions.

Exploration-Exploitation Dilemma

Exploitation maximizes reward on a one-step horizon. Exploration may produce greater cumulative reward in the long run. The two cannot be pursued simultaneously; any single action either exploits or explores.

No universally optimal balance exists without strong assumptions. Most practical methods incorporate heuristics or theoretically motivated exploration bonuses.

8 Action-Value Methods

8.1 Sample-Average Estimator

The simplest estimator averages all rewards received for action a:

Sample-Average Method

$$Q_t(a) = \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbf{1}_{A_i=a}}{\sum_{i=1}^{t-1} \mathbf{1}_{A_i=a}}.$$

By the law of large numbers, $Q_t(a) \to q_*(a)$ as the denominator $\to \infty$.

8.2 Action Selection Rules

Greedy selection:

$$A_t = \arg\max_a Q_t(a).$$

Always exploits; zero exploration. Can converge to a suboptimal action.

ε-greedy selection: with probability $1-\varepsilon$ select the greedy action; with probability ε select uniformly at random from all k actions. Formally:

$$A_t = \begin{cases} \arg\max_a Q_t(a) & \text{with probability } 1-\varepsilon, \\ \text{uniform sample from } \mathcal{A} & \text{with probability } \varepsilon. \end{cases}$$

As $t \to \infty$, every action is sampled infinitely often, guaranteeing $Q_t(a) \to q_*(a)$ for all a.

Remark. Larger ε explores more aggressively; smaller ε exploits more. If reward variance is zero (deterministic actions), $\varepsilon = 0$ works: a single trial identifies the best action and greedy is optimal. In **nonstationary** environments, continued exploration is necessary even with zero variance, because $q_*(a)$ can change over time.

8.3 The 10-Armed Testbed

A standard benchmark: 2000 randomly generated bandit instances, each with $k = 10$ arms. For each instance, $q_*(a) \sim \mathcal{N}(0,1)$ and $R_t \mid A_t = a \sim \mathcal{N}(q_*(a), 1)$. Methods are evaluated on average reward and percentage of times the optimal action is chosen, averaged over all runs and steps.

Key empirical findings:

- Greedy ($\varepsilon = 0$) converges fast initially but levels off below optimal because it locks onto a suboptimal action.
- $\varepsilon = 0.1$ explores more and eventually achieves a higher percentage of optimal selections.
- $\varepsilon = 0.01$ is intermediate: slower to improve but ultimately closer to optimal than $\varepsilon = 0.1$ (less wasted exploration at steady state).
- With higher reward variance, ε-greedy methods outperform greedy by an even larger margin.

9 Incremental Implementation

9.1 Efficient Update Rule

Recomputing sample averages from scratch at each step costs $O(n)$ time and memory. A constant-time, constant-memory update is derived as follows. Let Q_n be the estimate after $n-1$ selections of a given action:

$$Q_{n+1} = \frac{1}{n}\sum_{i=1}^{n} R_i = \frac{1}{n}(R_n + (n-1)Q_n) = Q_n + \frac{1}{n}[R_n - Q_n]\,.$$

Incremental Update (General Form)

$$\text{NewEstimate} \leftarrow \text{OldEstimate} + \alpha\,[\text{Target} - \text{OldEstimate}],$$

where $\alpha = 1/n$ for the sample average. The term [Target − OldEstimate] is the **error**: it is reduced by stepping toward the target. This pattern recurs throughout RL.

Only Q_n and n need to be stored; each update requires three arithmetic operations.

9.2 Complete Simple Bandit Algorithm

Algorithm 1 Simple Bandit (ε-greedy, sample average)

Initialize: $Q(a) \leftarrow 0$, $N(a) \leftarrow 0$ for all $a \in \{1, \ldots, k\}$
loop
  With probability $1 - \varepsilon$: $A \leftarrow \arg\max_a Q(a)$ (break ties randomly)
  With probability ε: $A \leftarrow$ uniform sample from $\{1, \ldots, k\}$
  $R \leftarrow \textsc{Bandit}(A)$
  $N(A) \leftarrow N(A) + 1$
  $Q(A) \leftarrow Q(A) + \dfrac{1}{N(A)}[R - Q(A)]$
end loop

10 Tracking a Nonstationary Problem

10.1 Constant Step-Size Parameter

In **nonstationary** environments, $q_*(a)$ drifts over time. Giving equal weight to all past rewards (sample average) is inappropriate; recent rewards should matter more. This is achieved by using a **constant** step-size α with $0 < \alpha \leq 1$:

$$Q_{n+1} = Q_n + \alpha[R_n - Q_n].$$

10.2 Exponentially Weighted Average

Unrolling the recurrence:

Exponentially Weighted (Recency-Biased) Average

$$Q_{n+1} = (1-\alpha)^n Q_1 + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} R_i.$$

The weight of reward R_i decays geometrically as $(1-\alpha)^{n-i}$, so more recent rewards have higher influence. This is a valid weighted average:

$$(1-\alpha)^n + \sum_{i=1}^{n} \alpha(1-\alpha)^{n-i} = 1.$$

10.3 Convergence Conditions

Let $\alpha_n(a)$ be the step-size at the n-th selection of action a. **Stochastic approximation theory** guarantees convergence of $Q_n \to q_*(a)$ with probability 1 if and only if:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty.$$

- **First condition**: steps must be large enough to eventually overcome any initial conditions or random fluctuations.
- **Second condition**: steps must eventually become small enough for the estimates to converge.

$\alpha_n = 1/n$ satisfies both conditions (second sum is $\pi^2/6$ by the Basel result). A constant α with $0 < \alpha \leq 1$ violates the second condition — estimates *never fully converge* but continue to track changes in $q_*(a)$, which is precisely the desirable behavior in nonstationary settings.

11 Optimistic Initial Values

All action-value methods are biased by the initial estimates $Q_1(a)$. This bias can be exploited constructively.

Optimistic initialization: set $Q_1(a) \gg \mathbb{E}[R]$ for all a (e.g., $Q_1(a) = 5$ when true values are near 0). Under greedy selection, any action selected yields a reward below the estimate, so the agent is disappointed and switches to a different action. This forces early exploration of all arms even without explicit randomization.

Properties of Optimistic Initialization

- Simple to implement; no extra hyperparameters beyond the initial value.
- Provides a natural way to encode prior knowledge about expected reward magnitude.
- Effective on **stationary** problems: exploration pressure fades as estimates converge.
- **Not suited for nonstationary problems**: the exploratory drive is temporary and cannot adapt to later changes in $q_*(a)$.

12 Upper Confidence Bound (UCB) Action Selection

12.1 Motivation

ε-greedy exploration selects non-greedy actions *uniformly at random*, ignoring structure: an action with a barely-below-greedy estimate and high uncertainty is treated identically to one that is clearly suboptimal. A principled improvement is to select among non-greedy actions according to their **potential** to be optimal, accounting for both the estimate and its uncertainty.

12.2 UCB Formula

UCB Action Selection

$$A_t = \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right],$$

where $N_t(a)$ is the number of times action a has been selected before time t, t is the total steps taken so far, and $c > 0$ controls exploration intensity. Any action with $N_t(a) = 0$ is treated as maximally uncertain and selected first.

12.3 Interpretation

The term $c\sqrt{\ln t \,/\, N_t(a)}$ is an **upper confidence bound** on $q_*(a)$: it is an optimistic estimate of how high the true value might be, derived from concentration-of-measure arguments (Hoeffding's inequality). Key properties:

- The bound **grows** with t (as more time passes, an un-tried action becomes increasingly suspicious).
- The bound **shrinks** with $N_t(a)$ (more evidence reduces uncertainty).
- Actions are tried according to how uncertain they are, not merely how infrequently.

Performance: UCB (with $c = 2$) typically outperforms ε-greedy on stationary testbeds.

Limitations: UCB is difficult to extend to the full RL setting with state-dependent value functions, especially with function approximation.

13 Gradient Bandit Algorithms

13.1 Preference-Based Action Selection

Instead of estimating action values, gradient bandits learn a **numerical preference** $H_t(a) \in \mathbb{R}$ for each action. Preferences have no direct interpretation as expected rewards; only relative differences matter. Action probabilities follow a **softmax** (Gibbs/Boltzmann) distribution:

Softmax Action Probabilities

$$\pi_t(a) = \Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}}.$$

Initially $H_1(a) = 0$ for all a, so all actions are equiprobable.

13.2 Stochastic Gradient Ascent Update

At step t, action A_t is drawn from π_t, reward R_t is observed, and preferences are updated by stochastic gradient ascent on $\mathbb{E}[R_t]$:

Gradient Bandit Update

$$H_{t+1}(A_t) = H_t(A_t) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)),$$
$$H_{t+1}(a) = H_t(a) - \alpha(R_t - \bar{R}_t)\,\pi_t(a), \quad \forall a \neq A_t,$$

where $\alpha > 0$ is the step-size and $\bar{R}_t$ is the running average of all rewards received up to time t, computed incrementally.

13.3 Role of the Baseline

$\bar{R}_t$ acts as a **reward baseline**:

- If $R_t > \bar{R}_t$: the probability of A_t *increases*; all other actions decrease.
- If $R_t < \bar{R}_t$: the probability of A_t *decreases*; all other actions increase.

The baseline does not affect the expected gradient (it is an unbiased estimate of $\mathbb{E}[R_t]$), but it **reduces variance** of the gradient estimate, leading to faster and more stable learning. Without the baseline ($\bar{R}_t = 0$), performance degrades significantly, particularly when the reward scale shifts (e.g., all $q_*(a)$ shifted by +4: without baseline, the algorithm must first discover the new scale, while with baseline it adapts immediately).

14 Comparative Summary

14.1 Algorithm Comparison

Method	Exploration mechanism	Key parameter	Notes
Greedy	None	—	Exploits only; can get stuck
ε-greedy	Random with prob. ε	$\varepsilon \in [0, 1]$	Simple, robust
Optimistic init.	Forced early exploration	$Q_1(a)$	Stationary only
UCB	Uncertainty bonus	$c > 0$	Principled; harder to extend
Gradient bandit	Softmax preferences	$\alpha > 0$	No value estimates needed

Each method has a parameter that controls the exploration-exploitation balance. Performance across all methods, measured as average reward over the first 1000 steps, forms a hump-shaped curve as a function of the parameter: too little exploration (small ε, c, α, or Q_0) causes suboptimal convergence; too much wastes reward.

14.2 Toward Full RL: Associative Search

Bandit problems are **non-associative**: a single, static situation. In general RL there are **multiple situations** and the agent must learn a mapping situation → action (a policy).

Contextual bandits (associative search) are an intermediate case: the agent observes a context (situation) at each step, which may determine which bandit instance is active. The agent must associate different actions with different contexts, combining trial-and-error learning with situation-action association. This is the bridge toward full sequential RL with delayed rewards and state transitions.

Part III

Markov Decision Processes

Markov Decision Processes (MDPs) provide the formal mathematical framework for sequential decision-making under uncertainty. Unlike the bandit setting, actions in an MDP affect not only the immediate reward but also the future state of the environment, introducing the problem of **delayed reward** and the need to trade off immediate versus long-term gains.

15 The Agent-Environment Interface

An MDP models the interaction between a **learner/decision-maker** (the agent) and the world it operates in (the environment). At each discrete time step $t = 0, 1, 2, \ldots$ the agent observes a state, selects an action, and receives a scalar reward.

MDP Tuple

A finite MDP is defined by:

- $\mathcal{S}$: finite set of states
- $\mathcal{A}$: finite set of actions (may depend on state: $\mathcal{A}(s)$)
- $\mathcal{R} \subset \mathbb{R}$: finite set of rewards
- **Dynamics function**:

$$p(s', r \mid s, a) \;=\; \Pr\{S_t = s',\, R_t = r \mid S_{t-1} = s,\, A_{t-1} = a\}$$

with normalization $\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1$ for all s, a.

The dynamics function p completely characterizes the environment. From it one derives:

$$p(s' \mid s, a) = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \qquad \text{(state-transition probabilities)}$$

$$r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r} r \sum_{s'} p(s', r \mid s, a) \qquad \text{(expected reward)}$$

$$r(s, a, s') = \sum_{r} r \, \frac{p(s', r \mid s, a)}{p(s' \mid s, a)} \qquad \text{(expected reward for transitions)}$$

Markov property. The state S_t must encode all information relevant for future decisions: $p(s', r \mid s, a)$ depends only on the current state and action, not on any earlier history. The trajectory takes the form

$$S_0,\; A_0,\; R_1,\; S_1,\; A_1,\; R_2,\; S_2,\; A_2,\; R_3,\; \ldots$$

Remark. The boundary between agent and environment is not physical but *functional*: it separates what the agent can control (actions) from what it cannot arbitrarily control (the rest). An agent may have complete knowledge of the environment dynamics and still face a non-trivial RL problem. This flexibility makes MDPs broadly applicable — the **art of RL** lies in choosing the right state representation.

15.1 Example: Recycling Robot

A mobile robot collects cans; its battery level is the only state. The MDP has:

- $\mathcal{S} = \{\text{high}, \text{low}\}$
- $\mathcal{A}(\text{high}) = \{\text{search}, \text{wait}\}$; $\mathcal{A}(\text{low}) = \{\text{search}, \text{wait}, \text{recharge}\}$

Transitions and rewards are parameterised by α (probability of staying high after searching from high), β (probability of staying low after searching from low), $r_s > r_w > 0$ (search and wait rewards), and a large penalty -3 for running out of battery.

s	a	s'	$p(s' \mid s, a)$	$r(s, a, s')$
high	search	high	α	r_s
high	search	low	$1-\alpha$	r_s
high	wait	high	1	r_w
low	search	high	$1-\beta$	-3
low	search	low	β	r_s
low	wait	low	1	r_w
low	recharge	high	1	0

16 Goals and Rewards

Reward Hypothesis

All goals and purposes of an agent can be formalized as the maximization of the expected value of the cumulative sum of a scalar signal called the *reward*.

The reward signal specifies **what** the agent should achieve, not **how**. For example:

- Chess: $+1$ for winning, 0 for draw, -1 for losing.
- Robot locomotion: $+1$ per step the robot moves forward.
- Resource management: reward proportional to profit.

Encoding prior knowledge in the reward (rather than in hand-crafted sub-goals) allows the agent to discover better strategies than those a human designer might anticipate.

17 Returns and Episodes

17.1 Episodic Tasks

In **episodic** tasks the interaction naturally breaks into episodes ending in a **terminal state** S_T. The return is simply the finite sum:

$$G_t \;=\; R_{t+1} + R_{t+2} + \cdots + R_T$$

17.2 Continuing Tasks and Discounted Return

In **continuing** tasks there is no terminal state. To keep the return finite, a **discount factor** $\gamma \in [0, 1)$ is introduced:

Discounted Return

$$G_t \;=\; \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;=\; R_{t+1} + \gamma G_{t+1}$$

For a constant reward $R_{t+k+1} = c$ the return is $G_t = c/(1-\gamma)$ (geometric series). Key interpretations:

- $\gamma \to 0$: **myopic** — only immediate reward matters.
- $\gamma \to 1$: **far-sighted** — all future rewards weighted nearly equally (requires finite sums).
- γ can be interpreted as the probability of survival at each step.

Example 17.1. CartPole. In the episodic formulation the agent receives $+1$ per step; an infinite-duration pole balance yields infinite return. The continuing formulation instead gives reward -1 on failure (pole falls at step K), leading to $G_0 = -\gamma^K$: the agent must survive as long as possible while discounting the inevitable termination penalty.

18 Policies and Value Functions

A **policy** is a mapping from states to probabilities over actions:

$$\pi(a \mid s) \;=\; \Pr\{A_t = a \mid S_t = s\}$$

RL algorithms specify how the policy changes with experience.

18.1 Value Functions

State-Value and Action-Value Functions

$$v_\pi(s) \;=\; \mathbb{E}_\pi[G_t \mid S_t = s] \;=\; \mathbb{E}_\pi\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s\right]$$

$$q_\pi(s,a) \;=\; \mathbb{E}_\pi[G_t \mid S_t = s,\, A_t = a]$$

Both v_π and q_π can be estimated from experience by averaging sampled returns.

18.2 Bellman Equation for v_π

Expanding the expectation using the law of total expectation and the recursive form of the return:

Bellman Expectation Equation

$$v_\pi(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma\, v_\pi(s')\Big]$$

This is a system of $|\mathcal{S}|$ linear equations with a **unique solution** v_π. The **backup diagram** for v_π shows the current state (open circle) branching to state-action pairs (filled circles) via π, then to successor states (open circles) via p.

Remark. The term *backup* refers to the transfer of value information from successor states back to the current state. Backup operators are the computational primitive underlying all DP and RL algorithms.

18.3 Gridworld Example

A 5×5 grid with actions $\{\mathrm{N}, \mathrm{S}, \mathrm{E}, \mathrm{W}\}$; off-grid transitions return reward -1 and leave the state unchanged. Special states: $A \to A'$ with reward $+10$; $B \to B'$ with reward $+5$. The value function under the equiprobable random policy with $\gamma = 0.9$ is obtained by solving the Bellman equations; values near A are highest due to the $+10$ teleport.

18.4 Golf Example

State = ball location; reward = -1 per stroke; hole = terminal state. The value function under a *putter-only* policy assigns negative values proportional to the expected number of strokes from each location. Locations near the hole have values close to -1.

19 Optimal Policies and Value Functions

19.1 Partial Ordering of Policies

A policy π is **at least as good as** π' if $v_\pi(s) \geq v_{\pi'}(s)$ for all $s \in \mathcal{S}$. There always exists at least one **optimal policy** π^* that is simultaneously better than or equal to all other policies.

Optimal Value Functions

$$v^*(s) \;=\; \max_\pi v_\pi(s) \qquad \forall s$$

$$q^*(s,a) \;=\; \max_\pi q_\pi(s,a) \;=\; \mathbb{E}[R_{t+1} + \gamma\, v^*(S_{t+1}) \mid S_t = s,\, A_t = a]$$

19.2 Bellman Optimality Equations

Bellman Optimality Equations

$$v^*(s) \;=\; \max_a \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma\, v^*(s')\Big]$$

$$q^*(s,a) \;=\; \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma \max_{a'} q^*(s', a')\Big]$$

These are **nonlinear** fixed-point equations (the max replaces the policy-weighted sum). They have a unique solution for finite MDPs with $\gamma < 1$ or guaranteed termination.

19.3 Extracting the Optimal Policy

- Given v^*: take the **greedy** action — one-step lookahead

$$\pi^*(s) \;=\; \arg\max_a \sum_{s',r} p(s', r \mid s, a)\Big[r + \gamma\, v^*(s')\Big]$$

- Given q^*: take the greedy action directly — **zero-step** lookahead

$$\pi^*(s) \;=\; \arg\max_a q^*(s,a)$$

  Knowing q^* means no environment model is needed at decision time.

19.4 Recycling Robot: Bellman Optimality Equations

$$v^*(h) = \max\begin{cases} \alpha\big[r_s + \gamma\, v^*(h)\big] + (1-\alpha)\big[r_s + \gamma\, v^*(l)\big] \\ r_w + \gamma\, v^*(h) \end{cases}$$

$$v^*(l) = \max\begin{cases} \beta\big[r_s + \gamma\, v^*(l)\big] + (1-\beta)\big[-3 + \gamma\, v^*(h)\big] \\ r_w + \gamma\, v^*(l) \\ \gamma\, v^*(h) \end{cases}$$

20 Optimality and Approximation

Solving the Bellman optimality equations exactly requires:

1. Accurate knowledge of the environment dynamics $p(s', r \mid s, a)$.
2. Sufficient computational resources (matrix inversion: $O(|\mathcal{S}|^3)$).
3. The Markov property to hold.

In practice these conditions rarely hold simultaneously. Backgammon, for instance, has approximately 10^{20} states — tabular methods are infeasible.

Tabular vs. Approximate Methods

- **Small state spaces** (tabular methods): store v or q in a table; guaranteed convergence to optimal.
- **Large state spaces** (function approximation): parameterise $v_\theta(s) \approx v^*(s)$ or $q_\theta(s, a) \approx q^*(s, a)$ using neural networks or other function classes; sacrifice exact optimality for tractability.

20.1 Categorization of RL Algorithms

RL algorithms can be organized along two orthogonal axes:

Category	**Value function V**	**Policy π**
Value-based	Explicit	Implicit (greedy)
Policy-based	None	Explicit
Actor-Critic	Explicit	Explicit

A second axis concerns the use of a **model** of the environment:

- **Model-based**: dynamics $p(s', r \mid s, a)$ are known or learned; planning is possible.
- **Model-free**: dynamics are unknown; the agent interacts directly and learns from experience.

Remark. The MDP framework introduced in this chapter is the foundation for all algorithms discussed in subsequent chapters. Tabular methods (dynamic programming, Monte Carlo, TD learning) will be covered first, followed by function approximation methods that scale to high-dimensional state spaces.

Part IV

Dynamic Programming

Dynamic Programming (DP) refers to a collection of algorithms that compute optimal policies given a perfect model of the environment (i.e., the full dynamics $p(s', r \mid s, a)$). DP methods are **model-based** and assume complete knowledge of the MDP. This is a strong requirement that limits their direct applicability, but DP is foundational: it provides the theoretical backbone on which model-free methods (Monte Carlo, TD learning) are built.

Key characteristics of DP:

- **High computational cost**, but polynomial in $|\mathcal{S}|$ and $|\mathcal{A}|$.

- Applied to **finite MDPs**. For continuous state/action spaces, a common approach is to **quantize** the spaces and apply DP to the resulting finite MDP.
- DP computes value functions by using the **Bellman equations as iterative update rules**, progressively improving value approximations.

21 Policy Evaluation (Prediction)

The **prediction problem** asks: given a fixed policy π, compute its state-value function v_π. This is also called **policy evaluation**.

21.1 Bellman Equation as Update Rule

Recall the Bellman expectation equation:

$$v_\pi(s) \;=\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big]$$

This is a system of $|\mathcal{S}|$ simultaneous linear equations. Rather than solving it directly (e.g., by matrix inversion), we use it as an **iterative update rule**. Starting from an arbitrary initial approximation v_0, we define the sequence:

$$v_{k+1}(s) \;\doteq\; \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, v_k(s')\big]$$

where v_k is the current approximation and v_{k+1} is the updated one. The fixed point of this iteration is v_π.

Convergence of Iterative Policy Evaluation

The sequence $\{v_k\}$ can be shown to **converge** to v_π as $k \to \infty$ under the conditions that guarantee the existence of v_π (either $\gamma < 1$ or guaranteed termination).

21.2 Iterative Policy Evaluation Algorithm

In practice the update is performed **in place**: a single array $V(s)$ is maintained, and each state is updated using the most recent values available (including values already updated in the current sweep).

Algorithm 2 Iterative Policy Evaluation (in-place), estimating $V \approx v_\pi$

Input: policy π to be evaluated
Parameter: threshold $\theta > 0$ (accuracy of estimation)
Initialize $V(s)$ for all $s \in \mathcal{S}^+$ arbitrarily, except $V(\text{terminal}) = 0$
repeat
  $\Delta \leftarrow 0$
  for each $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big]$
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
  end for
until $\Delta < \theta$

Complexity: $O(|\mathcal{S}|^2 \cdot |\mathcal{A}|)$ per sweep.

21.3 Example: Gridworld

Consider a 4×4 grid with nonterminal states $\mathcal{S} = \{1, 2, \ldots, 14\}$ and two terminal states (top-left and bottom-right corners). Actions $\mathcal{A} = \{\text{up}, \text{down}, \text{left}, \text{right}\}$; the transition model is deterministic; reward $R_t = -1$ on all transitions (undiscounted episodic task).

Under the equiprobable random policy ($\pi(a \mid s) = 0.25$ for all a), iterative policy evaluation produces a sequence of value functions $v_0, v_1, v_2, \ldots$ converging to v_π. As k increases, the greedy policy with respect to v_k quickly stabilizes to the optimal policy even before v_π is fully converged — for instance, iterations beyond $k = 3$ no longer change the greedy policy in this example.

22 Policy Improvement

Given the value function v_π for a policy π, we want to determine whether a **better policy** exists by changing the action in some state.

22.1 The Improvement Criterion

Consider selecting a new action a in state s and then following π thereafter. The value of this strategy is the **action-value function**:

$$q_\pi(s, a) \;=\; \sum_{s', r} p(s', r \mid s, a)\big[r + \gamma\, v_\pi(s')\big]$$

If $q_\pi(s, a) \geq v_\pi(s)$, then selecting a in state s and following π thereafter is at least as good as following π entirely. In this case, action a should be **substituted** for the action prescribed by $\pi(s)$.

22.2 Policy Improvement Theorem

Theorem 22.1 (Policy Improvement)**.** *Let π and π' be any pair of deterministic policies such that for all $s \in \mathcal{S}$:*

$$q_\pi(s,\, \pi'(s)) \;\geq\; v_\pi(s)$$

Then π' is at least as good as π:

$$v_{\pi'}(s) \;\geq\; v_\pi(s) \qquad \forall\, s \in \mathcal{S}$$

*If the first inequality is **strict** at any state, then the second inequality must be strict at at least one state.*

Proof (sketch). Starting from $v_\pi(s) \leq q_\pi(s, \pi'(s))$ and expanding using the Bellman equation, we repeatedly substitute $v_\pi(S_{t+1}) \leq q_\pi(S_{t+1}, \pi'(S_{t+1}))$:

$$\begin{aligned}
v_\pi(s) \;&\leq\; q_\pi(s, \pi'(s)) \\
&= \mathbb{E}[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s,\, A_t = \pi'(s)] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma\, v_\pi(S_{t+1}) \mid S_t = s] \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma\, q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\
&= \mathbb{E}_{\pi'}[R_{t+1} + \gamma\, R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s] \\
&\leq\; \cdots \\
&\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \mid S_t = s] \\
&= v_{\pi'}(s).
\end{aligned}$$

□

22.3 Greedy Policy Construction

Policy improvement extends the single-state idea to **all states and actions**, constructing a new **greedy** policy π':

$$\pi'(s) \doteq \arg\max_a q_\pi(s,a) = \arg\max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma\, v_\pi(s')\big]$$

By construction, π' satisfies $q_\pi(s,\pi'(s)) \geq v_\pi(s)$ for all s, so by the policy improvement theorem it is at least as good as π.

Optimality Condition

If $v_\pi = v_{\pi'}$ (the greedy policy does not improve), then v_π satisfies the Bellman optimality equation:

$$v_{\pi'}(s) = \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma\, v_{\pi'}(s')\big]$$

and both π and π' are **optimal policies**. This theory extends to stochastic policies $\pi(a \mid s)$.

23 Policy Iteration

The **Policy Iteration** algorithm alternates between policy evaluation and policy improvement, producing a sequence of monotonically improving policies and value functions:

$$\pi_0 \xrightarrow{\text{E}} v_{\pi_0} \xrightarrow{\text{I}} \pi_1 \xrightarrow{\text{E}} v_{\pi_1} \xrightarrow{\text{I}} \pi_2 \xrightarrow{\text{E}} \cdots \xrightarrow{\text{I}} \pi_* \xrightarrow{\text{E}} v_*$$

where $\xrightarrow{\text{E}}$ denotes policy evaluation and $\xrightarrow{\text{I}}$ denotes policy improvement.

Key Properties of Policy Iteration

- **Convergence** in a finite number of iterations is guaranteed for finite MDPs, since the number of deterministic policies is finite ($|\mathcal{A}|^{|\mathcal{S}|}$) and each iteration strictly improves the policy (unless already optimal).
- Each policy evaluation step is **initialized** with the value function from the previous policy, which greatly accelerates convergence.

23.1 Policy Iteration Algorithm

24 Value Iteration

Policy iteration requires a full policy evaluation (itself an iterative procedure) at each step. The **Value Iteration** algorithm truncates policy evaluation to a **single sweep**, combining it with policy improvement in one update rule.

24.1 Update Rule

The value iteration update combines policy improvement and single-step policy evaluation:

$$v_{k+1}(s) \doteq \max_a \sum_{s',r} p(s',r \mid s,a)\big[r + \gamma\, v_k(s')\big]$$

Algorithm 3 Policy Iteration (using iterative policy evaluation), estimating $\pi \approx \pi_*$

1. Initialization
$V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$

2. Policy Evaluation
repeat
  $\Delta \leftarrow 0$
  for each $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_{s',r} p(s', r \mid s, \pi(s))\big[r + \gamma\, V(s')\big]$ ▷ Deterministic policy assumed
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
  end for
until $\Delta < \theta$

3. Policy Improvement
policy-stable $\leftarrow$ **true**
for each $s \in \mathcal{S}$ **do**
  old-action $\leftarrow \pi(s)$
  $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big]$
  if *old-action* $\neq \pi(s)$ **then**
    policy-stable $\leftarrow$ **false**
  end if
end for
if *policy-stable* **then**
  stop and return $V \approx v_*$ and $\pi \approx \pi_*$
else
  go to **2**
end if

This is equivalent to turning the **Bellman optimality equation** into an update rule. The policy evaluation step of policy iteration can be truncated in several ways without losing convergence guarantees; value iteration is the extreme case of stopping after a single backup.

Value Iteration vs Policy Evaluation

The value iteration update is identical to the policy evaluation update, except that it takes the **maximum** over all actions rather than the expectation under a fixed policy. Value iteration does not explicitly represent or improve the policy — the policy is extracted from the converged value function at the end.

24.2 Value Iteration Algorithm

Algorithm 4 Value Iteration, estimating $\pi \approx \pi_*$

Parameter: threshold $\theta > 0$ (accuracy of estimation)
Initialize $V(s)$ for all $s \in \mathcal{S}^+$ arbitrarily, except $V(\text{terminal}) = 0$
repeat
  $\Delta \leftarrow 0$
  for each $s \in \mathcal{S}$ **do**
    $v \leftarrow V(s)$
    $V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big]$
    $\Delta \leftarrow \max(\Delta,\ |v - V(s)|)$
  end for
until $\Delta < \theta$

Output deterministic policy $\pi \approx \pi_*$ such that:
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma\, V(s')\big]$

Complexity: $O(|\mathcal{S}|^2 \cdot |\mathcal{A}|)$ per sweep (same as policy evaluation).

25 Asynchronous Dynamic Programming

Standard DP performs **synchronous** sweeps: each iteration updates every state exactly once. **Asynchronous DP** relaxes this requirement by ordering updates to let value information propagate from state to state more efficiently.

- Some states may not need their values updated as often as others.
- States irrelevant to optimal behavior can be skipped entirely.
- The agent's experience can guide which states to update (e.g., update states as the agent visits them).
- The update can be focused on the parts of the state set that are most relevant.

Asynchronous DP can **improve performance** in general and allows DP algorithms to be applied to domains with **large state spaces**, where full synchronous sweeps are prohibitively expensive.

26 Generalized Policy Iteration

Generalized Policy Iteration (GPI)

Generalized Policy Iteration (GPI) refers to the general schema of letting policy evaluation and policy improvement interact, regardless of the granularity or scheduling of each process. Both policy iteration and value iteration are instances of GPI:

- **Policy iteration**: full evaluation to convergence, then one improvement step.
- **Value iteration**: single evaluation sweep, interleaved with improvement.
- **Asynchronous DP**: partial, state-selective updates.

In all cases, evaluation pushes the value function toward v_π (consistency with the current policy), while improvement pushes the policy toward greedy with respect to the current value function. These two processes **compete** (evaluation targets a moving policy; improvement changes the policy the value function is tracking) yet **cooperate** (both drive toward the optimum). At the fixed point where neither process induces change, both the Bellman equation and the Bellman optimality equation are satisfied: $v = v_\pi = v_*$ and $\pi = \pi_*$.

27 Efficiency of Dynamic Programming

DP may not be practical for very large problems, but it is remarkably **efficient** compared to alternatives:

- DP methods are **polynomial** in $|\mathcal{S}|$ and $|\mathcal{A}|$, requiring fewer operations than some polynomial function of these quantities per iteration.
- The total number of deterministic policies to search is $|\mathcal{A}|^{|\mathcal{S}|}$, exponential in $|\mathcal{S}|$; DP avoids exhaustive enumeration.
- **Linear programming** methods become impractical at a much smaller number of states than DP methods.
- In practice, DP methods can solve MDPs with **millions of states**; both policy iteration and value iteration are used routinely.
- It is not clear which of the two algorithms is universally better; both are **faster than their theoretical worst-case run times** when started with good initial value functions or policies.

Bootstrapping

DP methods **update estimates** of the values of states **based on estimates** of the values of **successor states**: they update estimates on the basis of other estimates. This property is called **bootstrapping** and is shared by all DP methods. Monte Carlo methods (Lecture 5) do *not* require a model and do *not* bootstrap; Temporal-Difference methods (Lecture 6) do *not* require a model but *do* bootstrap.

Part V

Monte Carlo Methods

Unlike in Dynamic Programming, Monte Carlo (MC) methods do **not assume complete knowledge of the environment** (i.e., they do not require a model of the dynamics $p(s', r \mid s, a)$). MC methods are **model-free RL methods**: they are the first learning methods for estimating value functions and discovering optimal policies without a model.

MC methods require only **experience**: sample sequences of states, actions, and rewards obtained from actual or simulated interactions with the environment.

- **Learning from actual experience** requires no prior knowledge of the environment.
- **Learning from simulated experience** only requires a model that generates sample transitions (much easier than having the full probability distribution over all possible transitions as required in DP).

MC methods solve RL problems by **averaging sample returns** over episodes. We assume experience is split into episodes. Values and policies are updated **after each episode** (not after each step, as in Temporal Difference methods). MC methods adapt the idea of **GPI** defined in DP methods, but:

- DP methods require the model of the dynamics;
- MC methods learn the value function from sample returns.

28 Monte Carlo Prediction

28.1 Setting

Given a policy π, we aim to compute its value function. Recall that the value of a state is its expected return (expected cumulative future discounted reward). The **main idea of MC** is to average the returns observed after visits to the state.

Given a set of episodes obtained following π and passing through state s, each occurrence of s in an episode is called a **visit** to s. State s may be visited multiple times in the same episode.

First-Visit vs Every-Visit MC

- **First-visit MC**: estimates $v_\pi(s)$ as the average of the returns following the **first visit** to s in each episode.
- **Every-visit MC**: averages the returns following **all visits** to s across all episodes.

28.2 First-Visit MC Prediction Algorithm

28.3 Convergence

Both first-visit and every-visit MC converge to $v_\pi(s)$ as the number of visits (or first visits) to s goes to infinity.

First-visit MC convergence (1940s):

- Each return is an **independent, identically distributed** estimate of $v_\pi(s)$ with finite variance.

Algorithm 5 First-Visit MC Prediction, estimating $V \approx v_\pi$

Input: policy π to be evaluated
Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$
Returns$(s) \leftarrow$ empty list, for all $s \in \mathcal{S}$
loop
  Generate an episode using π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if S_t does not appear in $S_0, S_1, \ldots, S_{t-1}$ **then**
      Append G to Returns(S_t)
      $V(S_t) \leftarrow$ average(Returns(S_t))
    end if
  end for
end loop

- **Law of large numbers**: the sequence of averages converges to the expected value.
- Each average is an **unbiased estimate**.
- The standard deviation of the error falls as $1/\sqrt{n}$, where n is the number of returns averaged.

Every-visit MC convergence (Singh and Sutton, 1996): the proof is less simple but the estimate also converges quadratically ($q = 2$) to $v_\pi(s)$.

28.4 Example: Blackjack

Although in Blackjack we have complete knowledge of the rules, we do *not* have the distribution p of next events (model of the dynamics). For example: player's sum is 14, he sticks — what is the probability of terminating with reward +1 as a function of the dealer's showing card? Very difficult to compute analytically.

All such probabilities must be computed in advance when DP methods are used. Generating the sample games required by MC methods is instead easy. **This happens surprisingly often in practice** and makes MC methods very useful.

28.5 Backup Diagram and No Bootstrapping

The backup diagram for MC estimation of $v_\pi(s)$:

- **Root**: the state node s to be updated.
- **Below**: the entire trajectory of transitions along a single episode, all the way to the terminal state.

Important Observation

The estimate for one state in MC methods does **not build upon the estimate of any other state**, as is the case in DP. **MC methods do not perform bootstrapping.** As a consequence, the computational expense of estimating the value of a single state is **independent of the number of states** — useful for online estimation or estimation of subsets of states.

29 MC Estimation of Action Values

29.1 Why Action Values?

Without a transition model, **state values are not sufficient** to determine a policy: to select the action leading to the target state we would need $p(s', r \mid s, a)$. In MC methods we must explicitly estimate the value of each action $q_\pi(s, a)$ in order to finally estimate q^*.

The MC methods are the same used for estimating state values, but focused on **state-action pairs**. A state-action pair (s, a) is *visited* in an episode if state s is visited and action a is taken in it. First-visit and every-visit MC converge quadratically to the true values as the number of visits to each state-action pair approaches infinity.

29.2 The Problem of Maintaining Exploration

Problem: many state-action pairs may never be visited. If the policy is deterministic, one will observe returns only for one of the actions from each state; estimates of the other actions will not improve with experience. We need to estimate values of all actions from each state — this is the **problem of maintaining exploration**.

Solution 1 — Exploring starts: specify that episodes start in a state-action pair and every pair has nonzero probability of being selected. This guarantees all pairs are eventually visited.

> **Problem with exploring starts:** they cannot be relied upon in general (e.g., when learning from a real environment, the agent cannot be arbitrarily reset to any state-action pair).
> **Most common alternative:** consider only stochastic policies with nonzero probability of selecting all actions in each state (e.g., ε-greedy policies).

30 Monte Carlo Control

30.1 MC-Based GPI

MC estimation can be used for **control** (optimal policy approximation) following the GPI approach:

- Maintain both an approximate policy and an approximate value function.
- The value function is altered to better approximate q_{π_k}.
- The policy is improved greedily w.r.t. the current value function.

Policy evaluation: performed using MC prediction (assuming an infinite number of episodes, hence obtaining the exact q_{π_k}).

Policy improvement: done by making the policy **greedy** w.r.t. the current value function. Since we have an action-value function, **no model is needed** to construct the greedy policy:

$$\pi_{k+1}(s) = \arg\max_a q_{\pi_k}(s, a)$$

By the policy improvement theorem, $q_{\pi_{k+1}}(s, \pi_{k+1}(s)) \geq q_{\pi_k}(s, \pi_k(s))$, so the policy is ensured to improve and converge to the optimal policy and value function.

> MC methods can find optimal policies given **only sample episodes** and no other knowledge of the environment.

Two unlikely assumptions we made:

1. **A1: Availability of infinite number of episodes.** Solution 1: determine the number of iterations to guarantee theoretical bounds (expensive). Solution 2: reduce iterations in evaluation — it works in practice (as in value iteration).
2. **A2: Exploring starts.** Removed in the following.

In MC it is natural to alternate between evaluation and improvement on an **episode-by-episode** basis.

30.2 Monte Carlo ES (Exploring Starts)

Algorithm 6 Monte Carlo ES (Exploring Starts), estimating $\pi \approx \pi_*$

Initialize $Q(s,a) \in \mathbb{R}$ arbitrarily, $\pi(s) \in \mathcal{A}(s)$ arbitrarily, for all s, a
Returns$(s,a) \leftarrow$ empty list, for all s, a
loop
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ with all pairs having nonzero probability
  Generate episode from S_0, A_0 using π
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if (S_t, A_t) does not appear in $(S_0, A_0), \ldots, (S_{t-1}, A_{t-1})$ **then**
      Append G to Returns(S_t, A_t)
      $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    end if
  end for
end loop

Convergence of MC ES

MC ES **cannot converge to any suboptimal policy**. If it did, the value function would eventually converge to the value function of that suboptimal policy, which would cause the policy to change. Convergence seems inevitable but **has not yet been formally proved**.

31 Monte Carlo Control without Exploring Starts

31.1 On-Policy vs Off-Policy

How can we avoid the unlikely assumption of exploring starts? Two approaches:

- **On-policy methods:** evaluate or improve the policy that is used to make decisions (and produce data). MC ES is an example of an on-policy method.
- **Off-policy methods:** evaluate or improve a policy *different* from that used to generate the data.

In on-policy control methods, the policy is in general **soft**, i.e., $\pi(s,a) > 0$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}$, but it is gradually shifted closer and closer to a deterministic optimal policy.

31.2 On-Policy MC Control with ε-Greedy Policies

The on-policy method uses ε-greedy policies: they choose the action with maximal estimated action value most of the time, but with probability ε they select an action at random. GPI is used, with first-visit MC methods to estimate the action-value function.

GPI does not require that the improved policy always be greedy; it only requires that it **move towards** a greedy policy. For any ε-soft policy π, any ε-greedy policy w.r.t. q_π is guaranteed to be **better than or equal to** π.

Algorithm 7 On-Policy First-Visit MC Control (for ε-soft policies), estimating $\pi \approx \pi_*$

Parameter: $\varepsilon > 0$; Initialize $Q(s,a) \in \mathbb{R}$ arbitrarily, Returns$(s,a) \leftarrow$ empty list
$\pi \leftarrow$ ε-greedy policy w.r.t. Q
loop
  Generate episode using π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if (S_t, A_t) not in earlier steps of episode **then**
      Append G to Returns(S_t, A_t)
      $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))
      $A^* \leftarrow \arg\max_a Q(S_t, a)$
      for all $a \in \mathcal{A}(S_t)$ **do**

$$\pi(a \mid S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

      end for
    end if
  end for
end loop

Convergence: the policy improvement theorem assures that any ε-greedy policy w.r.t. q_π is an improvement over any ε-soft policy π. Let π' be the ε-greedy policy; then for each state s:

$$q_\pi(s, \pi'(s)) \geq v_\pi(s) \quad \Rightarrow \quad v_{\pi'}(s) \geq v_\pi(s)$$

Thus $\pi' \geq \pi$. The equality holds only when both policies are optimal among the ε-soft policies.

32 Off-Policy Prediction via Importance Sampling (Hints)

32.1 The On-Policy Dilemma and Off-Policy Solution

Dilemma of learning control methods: they seek to learn action values conditional on subsequent optimal behaviour, but they need to behave non-optimally to explore all actions and find optimal ones. The on-policy approach is a compromise: it learns action values not for the optimal policy, but for a **near-optimal policy** (e.g., ε-greedy) that still explores.

Off-policy solution: use two policies:

- **Target policy π:** the learned policy, which becomes the optimal policy.
- **Behavior policy b:** the exploratory policy used to generate data.

Learning is from data "off" the target policy $\Rightarrow$ **off-policy learning**.

Off-policy methods are of greater variance and slower to converge, but more powerful and general — they include on-policy methods as a special case (target = behavior). Additional uses include learning from data generated by non-learning controllers or human experts.

32.2 Importance Sampling

Almost all off-policy methods utilize **importance sampling**: a general technique for estimating expected values under one distribution given samples from another. Returns are weighted according to the **relative probability** of their trajectories occurring under target and behavior policies.

Given a starting state S_t, the probability of the subsequent trajectory occurring under any policy π is:

$$\Pr\{A_t, S_{t+1}, A_{t+1}, \dots, S_T \mid S_t, A_{t:T-1} \sim \pi\} = \prod_{k=t}^{T-1} \pi(A_k \mid S_k)\, p(S_{k+1} \mid S_k, A_k)$$

Importance Sampling Ratio

$$\rho_{t:T-1} = \frac{\prod_{k=t}^{T-1} \pi(A_k \mid S_k)\, p(S_{k+1} \mid S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k \mid S_k)\, p(S_{k+1} \mid S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

The ratio depends **only on the two policies and the sequence**, not on the MDP (the transition probabilities cancel).

The returns G_t generated by b have the wrong expectation ($\mathbb{E}_b[G_t \mid S_t = s] = v_b(s) \neq v_\pi(s)$), so they cannot be averaged directly. The importance sampling ratio transforms the return so that it has the correct expectation:

$$\mathbb{E}[\rho_{t:T-1}\, G_t \mid S_t = s] = v_\pi(s)$$

Two estimators:

- **Ordinary importance sampling:** unbiased but can have **infinite variance**.
- **Weighted importance sampling:** biased (but consistently), **dramatically lower variance** — strongly preferred in practice.

33 Summary: Monte Carlo Methods

Four Advantages of MC over DP

1. No model of the environment is required.
2. They can be used with **simulators** of the environment.
3. They can **focus on a subset of states** (scaling to large problems).
4. They **do not bootstrap**, hence they may be less harmed by violations of the Markov property.

Problem of maintaining sufficient exploration:

- **Exploring starts:** valid only for simulated episodes.

- **On-policy prediction/control:** not completely precise (converges to best ε-soft policy, not optimal).
- **Off-policy prediction/control:** the best method but more complex (requires target/behavior policy and importance sampling).

Part VI

Temporal-Difference Learning

Temporal Difference (TD) is the **most central and novel idea of RL**. TD is a combination of DP and MC ideas:

- Like MC, TD can **learn from raw experience**, without a model of the dynamics.
- Like DP, TD performs **bootstrapping**: it updates estimates based on other learned estimates without waiting for the final outcome.

The relationships between DP, MC and TD is a recurring theme in RL — these ideas blend into each other and can be combined. All three use the GPI approach but differentiate in how they solve the **prediction problem**.

34 TD Prediction

34.1 From MC to TD(0)

Both TD and MC use experience to solve the prediction problem. Given some experience following policy π, both methods update their estimate V of v_π for non-terminal states S_t.

MC waits until the return following the visit is known. Every-visit MC update (constant-α MC):

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{G_t}_{\text{MC target}} - V(S_t)]$$

where G_t is the actual return following time t. One must wait until the *end of the episode*.

TD methods wait only until the **next time step**. At time $t+1$ they immediately form a target and make a useful update using R_{t+1} and $V(S_{t+1})$.

TD(0) Update Rule (One-Step TD)

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)]$$

The update is performed immediately on transition to S_{t+1}.

TD(0) is a special case of TD(λ) (Ch. 12) and n-step TD (Ch. 7).

34.2 TD as Combination of MC and DP

Method	Target
MC	G_t (actual return — estimates expected value via sampling)
DP	$\sum_{s',r} p(s', r\|s, a)[r + \gamma V(s')]$ (full expectation — estimates V via bootstrapping)
TD	$R_{t+1} + \gamma V(S_{t+1})$ (combines both: samples *and* bootstraps)

- The **MC target** is an estimate because the expected value is unknown — a sample return is used in place of the true expected return.
- The **DP target** is an estimate because v_π is not known and the current estimate $V(S_{t+1})$ is used instead (though the expected values are completely provided by the model).
- The **TD target** is an estimate for *both* reasons: it samples the expected values **and** uses the current estimate V instead of the true v_π.

Sample updates (used by MC and TD) differ from **expected updates** (used by DP): sample updates are based on a single sample successor rather than a complete distribution of all possible successors.

34.3 TD Error

TD Error

$$\delta_t \;=\; R_{t+1} + \gamma\, V(S_{t+1}) - V(S_t)$$

A sort of error measuring the difference between the estimated value of S_t (i.e., $V(S_t)$) and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This error is based on the estimate made at time t and is not available until one time step later — the error in $V(S_t)$ is available at time $t+1$.

34.4 Example: Driving Home

In the driving-home analogy (Sutton & Barto), at each point during the drive we update our estimate of total travel time. MC would only update estimates after arriving home (G_t known at the end). TD updates at each step using the current elapsed time plus the revised estimate for the remaining journey. Each error is proportional to the **temporal difference in predictions**.

34.5 TD(0) Algorithm

Algorithm 8 Tabular TD(0), estimating $V \approx v_\pi$

Input: policy π; step-size $\alpha \in (0, 1]$
Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}^+$; $V(\text{terminal}) = 0$
for each episode **do**
  Initialize S
  repeat
    $A \leftarrow$ action given by π for S
    Take action A; observe R, S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal
end for

35 Advantages of TD Prediction Methods

TD methods have three main advantages over MC and DP:

1. **No model required:** TD methods do not require a model of the environment, unlike DP.

2. **Online and fully incremental:** TD methods wait only one time step to make an update, unlike MC which must wait until the end of the episode.
   - Some applications have very long episodes, making the wait too long.
   - Other applications are **continuing tasks** with no episode at all — MC cannot be applied, TD can.
3. **Less susceptible to slow learning:** MC methods have slow learning in some conditions (highly correlated episodes, infrequent terminal states); TD methods learn from each transition regardless of subsequent actions.

35.1 Convergence

For any fixed policy π, TD(0) has been proved to converge to v_π:

- **In the mean** for a constant step-size $\alpha_n = \alpha$ if it is sufficiently small.
- **With probability 1** if the step-size decreases according to stochastic approximation conditions:

$$\sum_{n=1}^{\infty} \alpha_n(a) = \infty \qquad \text{and} \qquad \sum_{n=1}^{\infty} \alpha_n^2(a) < \infty$$

Most convergence proofs apply only to the **tabular** case; some also apply to the case of general **linear function approximations** (Ch. 9).

35.2 Convergence Speed

Both TD and MC methods converge asymptotically to correct predictions. Which converges faster? This is an **open question** with no mathematical proof. **In practice**, TD methods usually converge faster than constant-α MC methods on stochastic tasks.

Example 35.1 (Random Walk)**.** A Markov Reward Process with 5 states and equal left/right transition probability. True values: $1/6, 2/6, 3/6, 4/6, 5/6$. TD(0) with $\alpha = 0.1$ learns values closer to the true values than constant-α MC after 100 episodes for most choices of α.

36 Batch Updating

36.1 Setting

Batch updating: suppose a finite amount of experience is available (e.g., 10 or 100 episodes). A common approach with incremental learning is to present the experience **repeatedly until convergence**:

1. Compute increments for every time step t where a non-terminal state is visited, but update the value function only **once after processing the complete batch**.
2. Repeat with the new value function until convergence.

Under batch updating, TD(0) converges **deterministically to a single answer** independent of α (as long as α is sufficiently small). Constant-α MC also converges deterministically, but to a **different answer**.

36.2 Why Different Answers?

Example 36.1 (You Are the Predictor)**.** Unknown Markov random process; 8 episodes observed. The only episode containing state A ended with $A, 0, B, 0$ (reward 0, then B, reward 0, terminal). State B was visited 8 times: 6 times terminated with reward +1, 2 with reward 0.

- **MC answer:** $V(A) = 0$ (seen A once, return was 0). **This minimizes MSE on the training data.**
- **TD answer:** $V(A) = 3/4$ (100% of times in A, process moved to B with reward 0, and $V(B) = 3/4$). **This models the Markov process.**

If the process is Markovian, the TD answer will produce lower error on **future data**, even though the MC answer is better on the *observed* data.

Batch MC vs Batch TD(0)

- **Batch MC** finds the estimate that **minimizes mean-squared error** on the training set, without considering the Markov property.
- **Batch TD(0)** finds the **certainty-equivalence estimate**: the estimate that would be exactly correct for the **maximum-likelihood model** of the Markov process — it considers the model of the process.

The **certainty-equivalence estimate** is the parameter value obtained assuming that the estimate of the underlying process was known with certainty. Batch TD(0) converges to it.

If n is the number of states, computing the certainty-equivalence estimate conventionally requires $O(n^2)$ memory and $O(n^3)$ computation. TD methods can approximate the same solution using memory no more than $O(n)$ and repeated computations over the training set. On tasks with large state spaces, **TD may be the only feasible way** of approximating the certainty-equivalence estimate.

37 SARSA: On-Policy TD Control

37.1 Setting

For the control problem, we use TD methods for evaluation (GPI approach). As with MC, we must trade off exploration and exploitation. We estimate the **action-value function** $q_\pi(s, a)$ for the policy π, adapting the TD method for state-action to state-action transitions:

SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]$$

Performed after every transition from a non-terminal state S_t. $Q(S_{T+1}, A_{T+1}) = 0$ if S_{T+1} is terminal.

The rule is called **SARSA** because it uses the elements $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$.

The on-policy control algorithm based on SARSA continually estimates q_π for the behavior policy π and at the same time changes π towards greediness w.r.t. q_π. There is **no explicit representation of the policy** — it is implicitly derived from the Q-function (selecting the action greedily on Q).

37.2 SARSA Algorithm

37.3 Convergence

SARSA converges with probability 1 to an optimal policy and action-value function as long as:

Algorithm 9 SARSA (on-policy TD control), estimating $Q \approx q_*$

Initialize $Q(s,a)$ for all s, a; $Q(\text{terminal}, \cdot) = 0$
for each episode **do**
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy) ▷ Exploration policy
  repeat
    Take action A; observe R, S'
    Choose A' from S' using policy derived from Q ▷ Exploration policy
    $Q(S,A) \leftarrow Q(S,A) + \alpha[R + \gamma\, Q(S',A') - Q(S,A)]$ ▷ Learns Q of exploration policy
    $S \leftarrow S'$; $A \leftarrow A'$
  until S is terminal
end for

- All state-action pairs are visited an **infinite number of times**.
- The policy converges in the limit to the greedy policy (e.g., ε-greedy with $\varepsilon = 1/t$).

Example 37.1 (Windy Gridworld)**.** Actions: up, down, left, right. Some columns have upward wind. Results with ε-greedy SARSA, $\varepsilon = 0.1$, $\alpha = 0.5$: the agent finds an effective policy (though not always optimal — after 8000 steps the policy takes 17 steps while the optimal takes 15). **Termination cannot be guaranteed in this task, so MC methods cannot easily be applied.**

38 Q-Learning: Off-Policy TD Control

38.1 The Q-Learning Update

Proposed by Watkins in 1989, Q-learning is one of the **early breakthroughs in RL**:

Q-Learning Update Rule

$$Q(S_t, A_t) \;\leftarrow\; Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big]$$

The learned action-value function Q **directly approximates** q^*, the optimal action-value function, **independent of the policy being followed** (off-policy). The policy however determines which state-action pairs are visited and updated.

This simplifies the analysis of the algorithm and enables early convergence proofs. For correct convergence it is only required that the followed policy ensures all pairs continue to be updated. Under this assumption and stochastic approximation conditions on step-size, Q converges with probability 1 to q^*.

38.2 Q-Learning Algorithm

Note: a' in the max is **not selected by the ε-greedy policy** as in SARSA.

38.3 Cliff Walking: SARSA vs Q-Learning

Example 38.1 (Cliff Walking)**.** Grid world, reward -1 on all transitions, -100 for stepping into the cliff. With ε-greedy action selection ($\varepsilon = 0.1$):

- **Q-learning** learns the **optimal path** (along the cliff edge).

Algorithm 10 Q-Learning (off-policy TD control), estimating $Q \approx q_*$

Initialize $Q(s, a)$ for all s, a; $Q(\text{terminal}, \cdot) = 0$
for each episode **do**
  Initialize S
  repeat
    Choose A from S using ε-greedy policy derived from Q ▷ Exploration policy
    Take action A; observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_{a'} Q(S', a') - Q(S, A)\big]$ ▷ Learns q^*, not exploration policy
    $S \leftarrow S'$
  until S is terminal
end for

- **SARSA** learns the **safe sub-optimal path** (detours around the cliff, because the ε-greedy policy occasionally takes it near the cliff edge during learning).

Q-learning has **worse online performance** during training (falls off the cliff more often). If ε is gradually reduced, **both methods converge to** q^*.

39 Maximization Bias and Double Learning

39.1 Maximization Bias

Control algorithms involve maximization in the construction of the target policy (e.g., max in Q-learning, ε-greedy in SARSA). A maximum over **estimated values** is used implicitly as an estimate of the maximum value. This can lead to a **positive bias** called maximization bias.

Example 39.1. Consider a single state s with many actions a having true values $q(s, a) = 0$, estimated values $Q(s, a)$ with uncertainty (some above, some below zero).

- The maximum of the **true values** $q(s, a)$ is 0.
- The maximum of the **estimates** is positive $\Rightarrow$ positive bias.

In the standard maximization bias example: from state A, the best policy is to select **right** (expected return 0, left gives -0.1). But TD methods may favor the **left** action from A because maximization bias makes B appear to have positive value.

39.2 Double Learning

Idea: divide the plays of each action into two sets and obtain two independent estimates Q_1 and Q_2.

- Use Q_1 to determine the maximizing action: $A^* = \arg\max_a Q_1(s, a)$.
- Use Q_2 to provide an estimate of its value: $Q_2(s, A^*) = Q_2(s, \arg\max_a Q_1(s, a))$.
- This estimate is **unbiased**: $\mathbb{E}[Q_2(s, A^*)] = q(s, A^*)$.

Observations:

1. We can repeat the process with the roles of Q_1 and Q_2 reversed to obtain a second unbiased estimate.
2. Only **one estimate is updated** on each step.

3. Double learning **doubles memory requirements**, but does not increase computation per step.

39.3 Double Q-Learning

Double Q-Learning Update

On each step, flip a coin. If heads, update Q_1:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma\, Q_2\Big(S_{t+1}, \arg\max_a Q_1(S_{t+1}, a)\Big) - Q_1(S_t, A_t)\Big]$$

If tails, do the same update with Q_1 and Q_2 switched (update Q_2). The behavior policy uses **both** action-value estimates. Double versions of SARSA and Expected SARSA also exist.

40 Expected SARSA (Hints)

Expected SARSA generalizes Q-learning by taking the **expectation over possible next actions** under the current policy π, rather than the maximum:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1})\, Q(S_{t+1}, a) - Q(S_t, A_t)\right]$$

- Given the next state S_{t+1}, this algorithm moves **deterministically** (no variance from sampling A_{t+1}).
- Eliminates the variance due to random selection of A_{t+1}.
- If π is the greedy policy, Expected SARSA reduces to Q-learning.

41 Summary: Temporal-Difference Learning

- TD is an alternative to MC for solving the prediction problem in GPI.
- **On-policy TD:** SARSA.
- **Off-policy TD:** Q-learning, Expected SARSA.
- A third way to use TD in control: **Actor-Critic methods** (Ch. 13 Sutton & Barto) — explicitly represent the policy, which is inferred from value functions in standard TD.
- TD methods are the **most widely used RL methods**.

In this lecture we analyzed **one-step, tabular, model-free** TD methods. The road ahead:

- **Ch. 7:** n-step TD (link to MC methods that perform all episode steps).
- **Ch. 8:** Model-based RL methods (link to planning) $\Rightarrow$ next lecture.
- **Part II:** Function approximation (link to deep RL).

Part VII

Planning and Learning with Tabular Methods

The goal of this part is to develop a **unified view** of:

- RL methods that require a model of the environment (e.g., dynamic programming, heuristic search) ⇒ **Model-based RL methods**, which rely on **planning**.
- RL methods that can be used without a model (e.g., MC, TD) ⇒ **Model-free RL methods**, which rely on **learning**.

Similarities: the heart of both approaches is the computation of value functions. All methods are based on:

1. Looking ahead to future events.
2. Computing a backed-up value.
3. Updating the approximate value function.

42 Models and Planning

42.1 Environment Models

Definition 42.1 (Model of the Environment)**.** A **model** of the environment is anything that the agent can use to predict how the environment will respond to its actions:

$$(S_t,\, A_t) \longrightarrow \text{Model} \longrightarrow (S_{t+1},\, R_{t+1})$$

If the model is stochastic, there are several possible next states and rewards with different probabilities; otherwise it is deterministic.

Distribution Models vs Sample Models

- **Distribution models:** produce a description of **all possibilities** and their probabilities. Stronger — can be used to generate samples. DP uses a distribution model $p(s', r \mid s, a)$.
- **Sample models:** produce **just one** of the possibilities, sampled according to probabilities. Usually easier to obtain. The Blackjack example used a sample model.

Example (sum of a dozen dice): it is easy to write a program that simulates dice rolls and computes the sum (sample model), but harder to enumerate all possible sums and their probabilities (distribution model).

Given a starting state and a policy:

- Sample model ⇒ produces a complete episode.
- Distribution model ⇒ produces all possible episodes and their probabilities.

The model is used to **simulate the environment** and produce **simulated experience**.

42.2 Planning

Definition 42.2 (Planning)**. Planning** is a computational process that takes a model as input and produces or improves a policy.

Two approaches to planning:

- **State-space planning:** search through the state space for an optimal policy or path to goal. Actions cause transitions between states; value functions are computed over states.
- **Plan-space planning:** search through the space of plans; operators transform plans to other plans; value functions are defined over the space of plans (e.g., evolutionary methods, partial-order planning).

Plan-space methods are **difficult to apply efficiently** to stochastic sequential decision problems (the target of RL). We focus on **state-space planning**.

42.3 Planning and Learning Share a Common Structure

All state-space planning methods:

1. Involve **computing value functions** as an intermediate step toward improving the policy.
2. Compute value functions by **updates/backup operations** applied to some experience.

Key difference: planning uses **simulated experience** (from the model); learning uses **real experience** (from the environment).

Since they share the same structure, ideas and algorithms can be **transferred** between planning and learning. Example: random-sample one-step tabular Q-planning converges to the optimal policy for the model, just as one-step tabular Q-learning converges to the optimal policy for the real environment.

43 Dyna: Integrated Planning, Acting, and Learning

43.1 Motivation

When planning is done **online**, information gained from interaction with the real environment can be used to **improve the model** used by planning itself.

Real experience has two roles in online planners:

1. **Model learning:** improve the model (make it more similar to the real environment).
2. **Direct RL:** improve the value function and policy directly.

Direct vs Indirect RL

Experience can improve value function and policy either **directly** (direct RL) or **indirectly** via the model (indirect RL = planning).

- **Indirect methods:** make fuller use of limited experience. Better policy with fewer environment interactions. Cognition, deliberative planning.
- **Direct methods:** simpler, not affected by biases in model design. Trial-and-error learning, reactive decision making.

43.2 Dyna-Q

Dyna-Q is a simple architecture integrating the major functions needed in an online planning agent. It includes:

- **Planning:** random-sample one-step tabular Q-planning.
- **Direct RL:** one-step tabular Q-learning.
- **Model learning:** table-based, for deterministic environments. After each transition $(S_t, A_t) \to (R_{t+1}, S_{t+1})$, the model records in its table entry for (S_t, A_t) the prediction that (R_{t+1}, S_{t+1}) will follow. When queried again with the same state-action pair, it returns the last-observed next-state and next reward. The Q-planning algorithm samples only from state-action pairs that have previously been experienced.

Algorithm 11 Tabular Dyna-Q

Initialize $Q(s,a)$ and Model(s,a) for all s, a
loop
(a) $S \leftarrow$ current (non-terminal) state
(b) $A \leftarrow \varepsilon$-greedy(S, Q)
(c) Execute A; observe R, S'
(d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_{a'} Q(S', a') - Q(S,A)\big]$ ▷ Direct RL
(e) Model$(S,A) \leftarrow R, S'$ ▷ Model learning (deterministic)
(f) **Repeat** n **times:** ▷ Planning
$\hat{S} \leftarrow$ random previously observed state
$\hat{A} \leftarrow$ random action previously taken in $\hat{S}$
$\hat{R}, \hat{S}' \leftarrow$ Model$(\hat{S}, \hat{A})$
$Q(\hat{S}, \hat{A}) \leftarrow Q(\hat{S}, \hat{A}) + \alpha\big[\hat{R} + \gamma \max_{a'} Q(\hat{S}', a') - Q(\hat{S}, \hat{A})\big]$
end loop

Steps (d), (e), (f) require little computation; **planning (step f) is the computationally intensive part**. If steps (e) and (f) are omitted, we obtain one-step tabular Q-learning.

> Planning proceeds **incrementally**, so it is trivial to intermix planning and acting. The agent is always both **reactive** (direct RL) and **deliberative** (planning). Learning and planning differ only in the **source** of their experience (real vs simulated).

43.3 Example: Dyna Maze

In a maze task, a Dyna-Q agent with $n = 5$ planning steps (n simulated Q-learning updates per real step) finds the goal much faster than a Q-learning agent with $n = 0$. With $n = 50$, performance improves even further — each real experience is exploited multiple times through planning.

44 When the Model is Wrong

Models may be incorrect because:

- The environment is stochastic and only a limited number of samples have been observed.
- The model was learned using function approximation that has generalized imperfectly.
- The **environment has changed** and its new behavior has not yet been observed.

When the model is incorrect, the planning process is likely to compute a **suboptimal policy**. In cases of an **optimistic model** (predicting greater reward or better transitions than are actually possible), the suboptimal policy may lead to discovery and correction of the modeling error.

Example 44.1 (Blocking Maze — Optimistic Model)**.** A shortcut initially exists, then gets blocked. Dyna-Q discovers the block and re-routes eventually. Dyna-Q+ (with exploration bonus) re-routes faster.

Example 44.2 (Shortcut Maze — Pessimistic Model)**.** The original path remains open, but a shorter path opens up later. Standard Dyna-Q never finds the shortcut (no incentive to explore). Dyna-Q+ does find it.

44.1 General Problem and Heuristic Solutions

This is another version of the **exploration-exploitation conflict**, but in planning:

- **Exploration** = trying actions that improve the model.
- **Exploitation** = behaving optimally given the current model.

No general solution exists, but simple **heuristics** work:

Dyna-Q+

Keep track for each state-action pair of **how many time steps have elapsed** since the pair was last tried in real interaction with the environment. The more time elapsed, the greater the chance the model is incorrect. A **bonus reward** is added on simulated experiences involving stale actions:

$$\hat{R} + \kappa\sqrt{\tau}$$

where τ is the time elapsed since the pair was last tried and κ is a small constant. This encourages re-testing long-untried actions.

45 Prioritized Sweeping

45.1 Motivation

In Dyna, simulated transitions are started in state-action pairs selected **uniformly at random** from all previously experienced pairs. Planning can be more efficient if simulated transitions and updates are **focused on particular state-action pairs**.

Example 45.1. In the second episode of a maze task, only the last state-action pair has positive value — performing model updates along almost all trajectories is pointless, as they take the agent from one zero-valued state to another (updates have no effect). As planning progresses, the region of useful updates grows, but planning is still less efficient than if focused where it can achieve the best improvements.

45.2 Backward Focusing

In real-world domains with large state spaces, unfocused search is very inefficient.

Strategy 1: backward from goal states. When the agent discovers a change (or incorrect transition) in the environment, it changes the estimated value of one state. The only useful one-step updates are those of **actions that lead directly into** the state whose value changed. This is **backward focusing**.

Strategy 2 (more general): backward from any state whose value changed. Not all predecessor state-action pairs are equally useful:

- States whose values have changed a lot are likely to have predecessors that also change a lot.
- In stochastic environments, variations in estimated transition probabilities also contribute to urgency.

Prioritized Sweeping

Prioritize updates according to a measure of their **urgency** and perform them in order of priority. Maintain a **priority queue** of state-action pairs whose estimated value would change non-trivially if updated, prioritized by the size of the expected change. Effects of changes are efficiently back-propagated.

Algorithm 12 Prioritized Sweeping (deterministic environments)

Initialize $Q(s,a)$, Model(s,a), priority queue $\mathcal{P}$
loop
  $S \leftarrow$ current non-terminal state; $A \leftarrow \pi(S)$
  Execute A; observe R, S'
  Model$(S,A) \leftarrow R, S'$
  $P \leftarrow |R + \gamma \max_{a'} Q(S',a') - Q(S,A)|$
  if $P > \theta$ **then**
    Insert (S,A) in $\mathcal{P}$ with priority P
  end if
  repeat
    $(\hat{S},\hat{A}) \leftarrow$ first element of $\mathcal{P}$
    $\hat{R}, \hat{S}' \leftarrow$ Model$(\hat{S},\hat{A})$
    $Q(\hat{S},\hat{A}) \leftarrow Q(\hat{S},\hat{A}) + \alpha[\hat{R} + \gamma \max_{a'} Q(\hat{S}',a') - Q(\hat{S},\hat{A})]$
    for all $(\bar{S},\bar{A})$ predicted to lead to $\hat{S}$ **do**
      $\bar{R} \leftarrow$ predicted reward for $(\bar{S},\bar{A},\hat{S})$
      $P \leftarrow |\bar{R} + \gamma \max_{a'} Q(\hat{S},a') - Q(\bar{S},\bar{A})|$
      if $P > \theta$ **then**
        Insert $(\bar{S},\bar{A})$ in $\mathcal{P}$ with priority P
      end if
    end for
  until $\mathcal{P}$ is empty or n updates done
end loop

Prioritized sweeping works also in **stochastic environments**: the model keeps counts of the number of times each state-action has been experienced and what the next states were; an **expected update** is used.

Limitation of expected updates in stochastic environments: they may waste computation on low-probability transitions. **Solution:** sample updates get closer to the true value function with less computation despite the variance introduced — they break the backing-up computation into small pieces focused on individual transitions.

45.3 Forward vs Backward Focusing

- **Backward focusing** (prioritized sweeping): update states whose predecessors' values have changed. Efficient for propagating discovered changes backward.

- **Forward focusing:** update states according to the **probability of being visited** under the current policy. Naturally generated by trajectory sampling.

46 Trajectory Sampling

46.1 Two Ways of Distributing Updates

1. **Sweeps through the entire state (or state-action) space:** classical DP approach — updating each state once per sweep, equal treatment of all states.
   - Problematic on large tasks: there may not be time to complete even one sweep.
2. **Sampling from the state/state-action space:**
   - *Uniformly* (as in Dyna-Q): same problems as exhaustive sweep for large problems.
   - *According to the on-policy distribution*: distribution observed when following the current policy. Easily generated by interacting with the model following the current policy ⇒ **trajectory sampling**.

46.2 Trajectory Sampling

Trajectory sampling: simulate explicit individual trajectories and perform updates at the state (state-action pair) encountered along the way.

Is the On-Policy Distribution a Good One?

Yes — at least better than the uniform distribution:
- E.g., if learning to play chess, you study positions that arise in real games, not random positions. Random positions may be valid but less probable, hence less important.
- On-policy distribution has significant advantages when **function approximation** is used.
- In general it achieves an **improvement in planning speed**.

Experimental comparison (uniform vs on-policy):

- Sampling on-policy gives a **large advantage for large problems**, in particular when a small subset of the state-action space is visited under the on-policy distribution.
- For small problems, uniform sampling may be faster early on.
- For large problems, on-policy trajectory sampling consistently outperforms uniform sampling.

47 Planning at Decision Time

47.1 Background Planning vs Decision-Time Planning

Two Modes of Planning

- **Background planning** (e.g., DP, Dyna): used to gradually improve a policy using simulated experience; completed before an action must be selected; focused on **all states**.
- **Decision-time planning:** started and completed **after encountering** a new state S_t; the output is the selection of a single action A_t; can look much deeper than one-step ahead; focused on the **particular current state**.

- Example of decision-time planning: selecting the action bringing to the (model-predicted) states with the highest value.
- Intermediate approaches: focus planning on the current state and **store the results** for later reuse.

48 Summary: Planning and Learning

RL as a Coherent Set of Ideas

All tabular RL methods (DP, MC, TD, Dyna, prioritized sweeping, trajectory sampling) share **three ideas in common**:

1. They all seek to **estimate value functions**.
2. They all **backup values** along actual or simulated state trajectories.
3. They all follow the **GPI strategy** (alternating policy evaluation and improvement).

The methods can be positioned along two dimensions:

	Sample updates	**Expected updates**
Low bootstrapping (MC-like)	MC methods	—
High bootstrapping (DP-like)	TD/Dyna	DP

- **Bootstrapping** axis: from Monte Carlo (full returns, no bootstrap) to TD(0) / DP (one-step bootstrap).
- **Update type** axis: sample updates (MC, TD, Dyna) vs expected updates (DP, Expected SARSA).
- Prioritized sweeping and trajectory sampling address the **order and focus** of updates.
- Dyna shows that **planning and learning are complementary** and can be seamlessly integrated in a single agent architecture.

Monte Carlo in the GPI Schema

Monte Carlo methods are still instances of **Generalized Policy Iteration** (GPI): they alternate between **prediction** (evaluating v_π or q_π from sampled returns) and **improvement** (acting greedily with respect to the current value estimates). The key difference is that *no model is needed at any point.*

49 Monte Carlo Prediction

49.1 Setting

The **prediction problem** asks: given a policy π, estimate the state-value function v_π. In MC prediction we do this by following π to generate episodes and averaging the observed returns.

An episode under policy π starting from state s takes the form:

$$S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T, S_T$$

The return from time t is $G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$.

Since $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$, we estimate $v_\pi(s)$ by averaging all returns observed from state s across many episodes.

49.2 First-Visit vs Every-Visit MC

A state s may appear multiple times within the same episode. Two variants exist:

First-Visit vs Every-Visit Monte Carlo Prediction

- **First-visit MC**: for each episode, only the **first** time s is visited contributes a return to the average. This is the more common variant.
- **Every-visit MC**: **every** visit to s within an episode contributes a return to the average.

Both converge to $v_\pi(s)$ as the number of visits $\to \infty$. First-visit MC has simpler theoretical analysis (i.i.d. returns); every-visit MC has lower variance but biased individual estimates.

49.3 First-Visit MC Prediction Algorithm

Algorithm 13 First-Visit MC Prediction, estimating $V \approx v_\pi$

Input: policy π to be evaluated
Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$
Initialize Returns$(s) \leftarrow$ empty list, for all $s \in \mathcal{S}$
loop
  Generate an episode using π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if S_t does not appear in $S_0, S_1, \ldots, S_{t-1}$ **then**
      Append G to Returns(S_t)
      $V(S_t) \leftarrow$ average(Returns(S_t))
    end if
  end for
end loop

Remark. The backward pass over the episode (from $t = T-1$ down to 0) allows computing G_t incrementally without storing all future rewards explicitly: $G_t = R_{t+1} + \gamma G_{t+1}$.

49.4 Convergence

By the **law of large numbers**, as the number of episodes $\to \infty$:

$$V(s) \;\to\; v_\pi(s) \qquad \forall\, s \text{ visited infinitely often.}$$

No approximation is involved in the target (complete returns), unlike bootstrapping methods, so MC prediction is **unbiased**. The variance of individual returns can be high, especially with many timesteps and large γ.

50 Monte Carlo Estimation of Action Values

50.1 Why Action Values?

When the model is **unknown**, estimating v_π alone is insufficient for policy improvement: to act greedily with respect to v_π, we would need to perform a one-step lookahead

$$\pi'(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$$

which requires knowing p. Instead, we estimate the **action-value function** $q_\pi(s, a)$ directly: the greedy action is then $\arg\max_a Q(s, a)$, requiring no model.

Advantage of q_π over v_π in Model-Free Settings

$$\pi'(s) = \arg\max_a Q(s, a)$$

No knowledge of $p(s', r \mid s, a)$ is needed. This is the principal reason MC (and TD) methods estimate q rather than v.

50.2 The Exploration Problem

A critical issue arises: if π is deterministic, many state-action pairs (s, a) may **never be visited**, making $q_\pi(s, a)$ impossible to estimate. The agent must ensure **sufficient exploration**.

Two main solutions:

1. **Exploring starts**: each episode begins with a randomly chosen state-action pair (s, a) with nonzero probability. All pairs are eventually visited.
2. **Soft policies**: the policy π always assigns nonzero probability to every action in every state ($\pi(a \mid s) > 0 \ \forall a, s$), e.g., ε-greedy policies.

51 Monte Carlo Control

51.1 GPI with MC

MC control follows the GPI schema, alternating between:

- **MC Policy Evaluation**: estimate q_{π_k} from observed episodes.
- **Policy Improvement**: set π_{k+1} to be greedy (or ε-greedy) with respect to q_{π_k}.

51.2 Monte Carlo with Exploring Starts

The simplest MC control algorithm assumes **exploring starts**: every state-action pair has a nonzero probability of being the starting pair of an episode.

Algorithm 14 Monte Carlo ES (Exploring Starts), estimating $\pi \approx \pi_*$

Initialize $Q(s,a)$ arbitrarily, $\pi(s) \in \mathcal{A}(s)$ arbitrarily, for all s, a
Initialize Returns$(s,a) \leftarrow$ empty list, for all s, a
loop
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ uniformly at random (exploring starts)
  Generate an episode from S_0, A_0 using π
  $G \leftarrow 0$
  for each step $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if (S_t, A_t) does not appear in $(S_0, A_0), \ldots, (S_{t-1}, A_{t-1})$ **then**
      Append G to Returns(S_t, A_t)
      $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    end if
  end for
end loop

Convergence of MC ES

MC ES converges to the optimal policy π^* and optimal action-values q^* **if** (i) exploring starts holds and (ii) episodes are sufficiently long. Exploring starts is an assumption that may be difficult to guarantee in real environments (e.g., a physical robot cannot be reset to any arbitrary state).

52 On-Policy Monte Carlo Control

52.1 On-Policy vs Off-Policy

- **On-policy** methods: the policy used to *generate data* is the *same* policy being improved. The agent evaluates and improves the behavior policy.
- **Off-policy** methods: data is generated by a **behavior policy** $b \neq \pi$, while the **target policy** π is improved separately.

On-policy methods are simpler but require **soft** policies (nonzero probability for all actions) to avoid the exploring-starts assumption.

52.2 ε-Soft Policies

Definition 52.1 (ε-soft policy)**.** A policy π is **ε-soft** if

$$\pi(a \mid s) \;\geq\; \frac{\varepsilon}{|\mathcal{A}(s)|} \qquad \forall\, s \in \mathcal{S},\ \forall\, a \in \mathcal{A}(s)$$

for some $\varepsilon > 0$. Every action is tried with at least some minimum probability.

The simplest ε-soft policy is ε-greedy: with probability $1-\varepsilon$ take the greedy action; with probability ε choose uniformly at random.

52.3 On-Policy MC Control (ε-greedy)

Algorithm 15 On-Policy First-Visit MC Control (for ε-soft policies), estimating $\pi \approx \pi_*$

Parameter: $\varepsilon > 0$
Initialize $Q(s,a) \in \mathbb{R}$ arbitrarily, for all s, a
Initialize Returns$(s,a) \leftarrow$ empty list, for all s, a
Initialize π as ε-greedy with respect to Q
loop
  Generate episode using π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    if (S_t, A_t) does not appear earlier in episode **then**
      Append G to Returns(S_t, A_t)
      $Q(S_t, A_t) \leftarrow$ average(Returns(S_t, A_t))
      $A^* \leftarrow \arg\max_a Q(S_t, a)$ (break ties randomly)
      for all $a \in \mathcal{A}(S_t)$ **do**

$$\pi(a \mid S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

      end for
    end if
  end for
end loop

52.4 GLIE: Convergence Condition

On-policy MC control with a fixed $\varepsilon > 0$ converges to the best ε-**soft** policy, not necessarily to the globally optimal policy. To recover the true optimum, the policy must satisfy:

GLIE — Greedy in the Limit with Infinite Exploration

A sequence of policies $\{\pi_k\}$ is **GLIE** if:

1. Every state-action pair is visited infinitely often: $\lim_{k\to\infty} N_k(s,a) = \infty$ for all s, a.
2. The policy converges to a greedy policy: $\lim_{k\to\infty} \pi_k(a \mid s) = \mathbf{1}[a = \arg\max_{a'} Q_k(s,a')]$.

A common GLIE schedule: $\varepsilon_k = 1/k$ — this satisfies both conditions. Under GLIE, MC control converges to the **optimal policy** π^*.

Key Trade-off

- Fixed ε: guarantees exploration at every step but converges only to the best ε-soft policy (not optimal).
- GLIE ($\varepsilon_k \to 0$): converges to the optimal policy but exploration decreases over time, which can slow learning in nonstationary environments.

53 Off-Policy Prediction via Importance Sampling

53.1 Motivation

In **off-policy** learning, the agent wishes to estimate v_π (or q_π) for a **target policy** π, while data is generated by a different **behavior policy** b.

Requirements:

- **Coverage**: $\pi(a \mid s) > 0 \Rightarrow b(a \mid s) > 0$ for all s, a. Every action taken by π must also be possible under b; otherwise, some returns would never be observed.
- b is typically more exploratory than π (e.g., b is ε-soft, π is greedy).

53.2 Importance Sampling Ratio

Returns generated under b have the wrong distribution. The **importance sampling ratio** corrects for this mismatch by reweighting returns. For a trajectory $A_t, S_{t+1}, A_{t+1}, \ldots, S_T$ starting from $S_t = s$:

Importance Sampling Ratio

$$\rho_{t:T-1} = \prod_{k=t}^{T-1} \frac{\pi(A_k \mid S_k)}{b(A_k \mid S_k)}$$

This is the likelihood ratio of the trajectory under π relative to b. It accounts for the different probabilities that the two policies assign to each action.

Note that transition probabilities $p(S_{k+1} \mid S_k, A_k)$ cancel in the ratio (they are the same under both policies). The ratio depends only on the policies, not the dynamics.

53.3 Ordinary and Weighted Importance Sampling

Let $\mathcal{J}(s)$ be the set of all time steps at which state s is first visited (across all episodes), and $T(t)$ the time of termination of the episode containing time step t. The target return weighted by the IS ratio is $\rho_{t:T(t)-1} G_t$.

Ordinary vs Weighted Importance Sampling

Ordinary IS (OIS):

$$V(s) = \frac{\sum_{t \in \mathcal{J}(s)} \rho_{t:T(t)-1}\, G_t}{|\mathcal{J}(s)|}$$

Weighted IS (WIS):

$$V(s) = \frac{\sum_{t \in \mathcal{J}(s)} \rho_{t:T(t)-1}\, G_t}{\sum_{t \in \mathcal{J}(s)} \rho_{t:T(t)-1}}$$

53.4 Bias-Variance Trade-off

Estimator	Bias	Variance
Ordinary IS	Unbiased	Can be **infinite** (unbounded ρ)
Weighted IS	Biased (converges asymptotically)	**Finite** — dramatically lower

Remark. In practice, **WIS is strongly preferred**: its bias diminishes to zero as the number of episodes grows, while OIS can exhibit extremely large variance (the product of ratios can be very large or very small), leading to unreliable estimates. The variance of OIS can be **infinite** even when the variance of individual returns is finite.

53.5 Intuition: What Importance Sampling Does

The IS ratio $\rho_{t:T-1}$ can be thought of as a weight that asks: "How likely would this trajectory have been under π, relative to how likely it was under b?" If π and b chose the same actions along the trajectory, $\rho = 1$. If π would have taken actions much more aggressively than b, $\rho \gg 1$ (amplifying the return). If π would have taken different actions, $\rho \approx 0$ (the return is discounted or discarded).

54 Incremental Implementation of Off-Policy MC

Maintaining lists of all observed returns and recomputing averages is memory-inefficient. Incremental updates allow constant-memory estimation.

54.1 Incremental Weighted Importance Sampling

For weighted IS, define the cumulative sum of weights C_n and the weighted average Q_n after n returns:

Incremental WIS Update

Initialize $Q(s,a) \leftarrow 0$, $C(s,a) \leftarrow 0$ for all s, a. After observing return G with IS weight W:

$$C(s,a) \leftarrow C(s,a) + W$$

$$Q(s,a) \leftarrow Q(s,a) + \frac{W}{C(s,a)}[G - Q(s,a)]$$

This generalizes the ordinary incremental update (Section 4) by replacing the uniform weight $1/n$ with the normalized IS weight W/C.

55 Off-Policy Monte Carlo Control

Off-policy MC control estimates q_* (and therefore π_*) from episodes generated by a behavior policy b. The target policy π is kept greedy throughout; exploration is delegated entirely to b.

Remark. The inner loop exits as soon as the greedy target policy disagrees with the action actually taken: at that point, $\pi(A_t \mid S_t) = 0$, so the IS ratio becomes zero and the return contributes nothing. Only the suffix of the episode during which b happened to agree with π is used for learning. This makes off-policy MC control **learn slowly** on long episodes — a limitation that Temporal-Difference methods overcome.

56 Summary: Monte Carlo vs Dynamic Programming

Property	Dynamic Programming	Monte Carlo
Model required	Yes (full $p(s', r \mid s, a)$)	No
Bootstrapping	Yes	No
Update trigger	Every sweep	End of episode
Bias	None (exact equations)	None (unbiased returns)
Variance	Low	High (long returns)
Scalability	Limited by $\lvert\mathcal{S}\rvert$	Scales with episode length
Exploration needed	No (model-based planning)	Yes (must visit all pairs)

Algorithm 16 Off-Policy MC Control, estimating $\pi \approx \pi_*$

Initialize $Q(s,a) \in \mathbb{R}$ arbitrarily, $C(s,a) \leftarrow 0$, for all s, a
$\pi(s) \leftarrow \arg\max_a Q(s,a)$ for all s (greedy target policy)
loop
  $b \leftarrow$ any soft policy (behavior policy)
  Generate episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$, $W \leftarrow 1$
  for $t = T-1, T-2, \ldots, 0$ **do**
    $G \leftarrow \gamma G + R_{t+1}$
    $C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
    $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \dfrac{W}{C(S_t, A_t)}[G - Q(S_t, A_t)]$
    $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$
    if $A_t \neq \pi(S_t)$ **then**
      exit inner loop (episode tail no longer consistent with π)
    end if
    $W \leftarrow W \cdot \dfrac{\pi(A_t \mid S_t)}{b(A_t \mid S_t)} = W \cdot \dfrac{1}{b(A_t \mid S_t)}$ (π is deterministic so $\pi(A_t \mid S_t) \in \{0, 1\}$)
  end for
end loop

Core Advantages of Monte Carlo Methods

- MC methods can learn from **actual interaction** with the environment, without a model.
- Value estimates for each state are **independent**: the estimate for s does not depend on the estimate for any other state s'. This eliminates the risk of **error propagation** from incorrect value estimates (a weakness of bootstrapping methods).
- MC methods naturally focus computation on **states that are actually visited**, making them efficient in problems with large or unknown state spaces.
- They work well with **simulations**: if a simulator is available, MC methods can generate as many episodes as needed without requiring an explicit model.

Limitations of Monte Carlo Methods

- **Episodic tasks only** (basic formulation): updates happen at episode end; not directly applicable to continuing tasks.
- **High variance**: returns accumulate randomness across many steps; requires many episodes to reduce estimation error.
- **Slow off-policy learning**: IS ratios can be very small or very large, especially over long episodes, causing high variance or near-zero updates.
- **Exploration**: without exploring starts or soft policies, many state-action pairs may never be visited.

The next lecture (**Temporal-Difference learning**) addresses the first two limitations by combining the model-free advantage of MC with the bootstrapping efficiency of DP.

Part VIII

Temporal-Difference Learning

Temporal-Difference (TD) learning is the central idea of modern reinforcement learning. It combines the best properties of Monte Carlo and Dynamic Programming:

- Like **Monte Carlo**: TD methods are **model-free** — they learn directly from raw experience without a model of the environment's dynamics.
- Like **Dynamic Programming**: TD methods **bootstrap** — they update estimates based on other learned estimates, without waiting for the episode to end.

The Central Idea of TD Learning

After each transition $(S_t, A_t, R_{t+1}, S_{t+1})$, TD methods update the value of S_t using the **immediately observed reward** R_{t+1} and the **current estimate** of the next state's value $V(S_{t+1})$, rather than waiting for the complete return G_t.

57 TD Prediction: TD(0)

57.1 The TD(0) Update Rule

In Monte Carlo prediction, the target for updating $V(S_t)$ is the actual return G_t:

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{G_t}_{\text{MC target}} - V(S_t)]$$

In **one-step TD** (TD(0)), the return $G_t = R_{t+1} + \gamma G_{t+1}$ is approximated by replacing G_{t+1} with the current estimate $V(S_{t+1})$:

TD(0) Update Rule

$$V(S_t) \leftarrow V(S_t) + \alpha[\underbrace{R_{t+1} + \gamma V(S_{t+1})}_{\text{TD target}} - V(S_t)]$$

The quantity $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the **TD error**: it measures the difference between the estimated value of S_t and a better estimate based on what actually happened.

The update happens **online, after every step** — no need to wait until the end of the episode. This makes TD(0) applicable to both episodic and continuing tasks.

57.2 TD(0) Prediction Algorithm

57.3 TD Error and the Bellman Equation

The TD target $R_{t+1} + \gamma V(S_{t+1})$ can be seen as a **sample** of the right-hand side of the Bellman expectation equation:

$$v_\pi(s) = \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]$$

Algorithm 17 Tabular TD(0), estimating $V \approx v_\pi$

Input: policy π to be evaluated; step-size $\alpha \in (0, 1]$; discount γ
Initialize $V(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\text{terminal}) = 0$
for each episode **do**
  Initialize S
  repeat
    $A \leftarrow$ action given by π for S
    Take action A; observe R, S'
    $V(S) \leftarrow V(S) + \alpha[R + \gamma V(S') - V(S)]$
    $S \leftarrow S'$
  until S is terminal
end for

TD(0) drives $V(S_t)$ toward this sampled target. Once $V = v_\pi$ exactly, the expected TD error is zero. Formally, the fixed point of TD(0) satisfies:

$$\mathbb{E}_\pi[\delta_t \mid S_t = s] = 0 \quad \Longleftrightarrow \quad V(s) = v_\pi(s)$$

Convergence of TD(0)

For any fixed policy π, tabular TD(0) converges to v_π with probability 1 under the standard stochastic approximation conditions on the step-size sequence $\{\alpha_t\}$:

$$\sum_t \alpha_t = \infty \qquad \text{and} \qquad \sum_t \alpha_t^2 < \infty$$

With a constant α, TD(0) converges *in the mean* to a neighborhood of v_π (does not converge exactly, but tracks nonstationary targets).

57.4 Example: Random Walk

Consider a Markov chain with states $\{A, B, C, D, E\}$ and two terminal states (left and right). All transitions are equally likely; rewards are 0 everywhere except $+1$ on reaching the right terminal state. The true values under random walk are $v(A) = 1/6, v(B) = 2/6, \ldots, v(E) = 5/6$.

TD(0) with $\alpha = 0.1$ converges substantially faster than MC (constant α MC) to the true values on this problem, because TD leverages the structure of the Markov chain online rather than waiting for episode completion.

58 Comparing TD, MC, and DP

58.1 Update Targets

The three families differ only in **what target** is used to update $V(s)$:

Method	**Target**	**Model?**	**Bootstraps?**
Monte Carlo	$G_t = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{T-t-1} R_T$ (complete return)	No	No
TD(0)	$R_{t+1} + \gamma V(S_{t+1})$ (one-step estimate)	No	Yes
DP	$\sum_{s',r} p(s', r \mid s, a)[r + \gamma V(s')]$ (expected one-step)	Yes	Yes

58.2 Bias-Variance Trade-off

- **MC**: unbiased (uses true returns), but **high variance** — returns accumulate noise over many random steps.
- **TD**: **biased** (bootstraps off imperfect V), but **low variance** — only one random step of noise per update.
- **DP**: also biased (bootstraps), but uses **expectations** over all transitions (no sampling noise), requiring a model.

Why TD is Often Faster in Practice

Even though TD is biased, its dramatically lower variance typically leads to faster learning than MC on most tasks. The bias decreases as V improves, and the low-variance property remains throughout training. On the random walk benchmark, TD consistently outperforms constant-α MC for all values of α.

58.3 Online vs Offline Updates

- **TD**: updates are **online** — one update per step, immediately after each transition. Can be applied to continuing tasks.
- **MC**: updates are **offline** — one update per state per episode, only after the episode ends.
- TD is therefore more suitable for **long or infinite episodes** and for settings where learning while acting matters.

58.4 Optimality: Batch TD vs Batch MC

In the **batch setting** (fixed dataset of episodes, update repeatedly until convergence), TD(0) and constant-α MC converge to **different** solutions:

- **Batch MC** converges to the estimates that **minimize mean squared error** on the training data.
- **Batch TD(0)** converges to the **maximum likelihood Markov model** estimate — the values that would be exactly correct *if* the MDP were as implied by the data. This exploits the Markov structure and typically generalizes better.

Certainty-Equivalence Estimate

Batch TD(0) finds the **certainty-equivalence estimate**: it fits the MLE transition model from data, then solves for the value function of that model exactly. This explains why TD often achieves better predictive accuracy than MC even with the same data, in environments that are well described by Markov chains.

59 SARSA: On-Policy TD Control

59.1 From Prediction to Control

To use TD learning for **control**, we must estimate **action values** $q_\pi(s, a)$ rather than state values $v_\pi(s)$ (to avoid needing a model at the improvement step). The TD(0) update for action values is:

SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma\, Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big]$$

The tuple $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ — **SARSA** — gives the algorithm its name. The next action A_{t+1} is selected by the current policy (e.g., ε-greedy), making this an **on-policy** method.

At a terminal state, $Q(S_{T+1}, \cdot) = 0$.

59.2 SARSA Algorithm

Algorithm 18 SARSA (on-policy TD control), estimating $Q \approx q_*$

Parameters: step-size $\alpha \in (0, 1]$; $\varepsilon > 0$; discount γ
Initialize $Q(s, a)$ arbitrarily for all s, a; $Q(\text{terminal}, \cdot) = 0$
for each episode **do**
  Initialize S
  Choose A from S using ε-greedy policy derived from Q
  repeat
    Take action A; observe R, S'
    Choose A' from S' using ε-greedy policy derived from Q
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma\, Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$
  until S is terminal
end for

59.3 Convergence of SARSA

Convergence of SARSA

SARSA converges to the **optimal policy** π^* and optimal action-values q^* with probability 1 provided:

1. All state-action pairs are visited infinitely often (GLIE policy, e.g., $\varepsilon_t \to 0$ appropriately).
2. The step-sizes satisfy $\sum_t \alpha_t = \infty$ and $\sum_t \alpha_t^2 < \infty$.

59.4 Example: Windy Gridworld

A 7×10 grid where columns 4–8 have upward wind that shifts the agent up by 1 or 2 rows per step. Reward is -1 per step until the goal. SARSA with $\varepsilon = 0.1$, $\alpha = 0.5$, $\gamma = 1$ learns an effective policy quickly; the cumulative reward curve shows steady improvement with no plateau.

60 Q-Learning: Off-Policy TD Control

60.1 The Q-Learning Update

Q-learning (Watkins, 1989) is the most famous TD control algorithm. Its key innovation is using the **maximum** action-value for the next state as the bootstrap target, regardless of which action is actually taken:

Q-Learning Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big]$$

The TD target $R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$ directly approximates $q^*(S_t, A_t)$, bypassing the need to evaluate the current policy.

This is an **off-policy** method: behavior (the action actually taken in the environment) can follow any policy (e.g., ε-greedy), while the **target policy** being learned is always greedy.

60.2 Q-Learning Algorithm

Algorithm 19 Q-Learning (off-policy TD control), estimating $Q \approx q_*$

Parameters: step-size $\alpha \in (0, 1]$; $\varepsilon > 0$; discount γ
Initialize $Q(s, a)$ arbitrarily for all s, a; $Q(\text{terminal}, \cdot) = 0$
for each episode **do**
  Initialize S
  repeat
    Choose A from S using ε-greedy policy derived from Q
    Take action A; observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_{a'} Q(S', a') - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal
end for

Convergence of Q-Learning

Q-learning converges to q^* with probability 1, given that all state-action pairs are visited infinitely often and standard step-size conditions hold. Because the target directly uses $\max_{a'} Q(S', a')$, convergence does **not** require the behavior policy to converge; any policy that maintains sufficient exploration works.

60.3 SARSA vs Q-Learning: The Cliff Walking Example

Example 60.1 (Cliff Walking)**.** A 4×12 gridworld: the agent starts at the bottom-left, must reach the bottom-right goal, and must avoid a cliff along the bottom edge (reward -100, return to start). All other transitions yield reward -1.

- **SARSA** (on-policy): learns to take a **safe path** that goes around the cliff. Because ε-greedy exploration causes occasional random actions, SARSA incorporates the risk of falling into its estimates and avoids the cliff edge.
- **Q-learning** (off-policy): learns the **optimal path** along the cliff edge (shortest path to goal), but during training it falls often because the exploratory ε-greedy behavior policy takes it near the cliff. Q-learning gets a lower reward per episode during training, but its learned policy is better.

On-Policy vs Off-Policy in Practice

- **Q-learning** learns the *optimal* policy, but the policy it executes during learning may be risky (exploration causes dangerous actions).
- **SARSA** learns the *best policy given the exploration constraints*: it is safer during training and preferred when online performance (not just asymptotic quality) matters.
- With $\varepsilon \to 0$, SARSA and Q-learning converge to the same optimal policy.

61 Expected SARSA

61.1 Motivation

SARSA uses the next action A_{t+1} actually sampled from π in the update, introducing variance. **Expected SARSA** removes this variance by taking the **expectation** over all next actions under π:

Expected SARSA Update Rule

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a \pi(a \mid S_{t+1})\, Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

The target is the **expected value** of $Q(S_{t+1}, \cdot)$ under the current policy, rather than a single sample.

- Expected SARSA is computationally more expensive per step than SARSA (must sum over all actions).
- It has **lower variance** than SARSA and generally converges faster.
- With the greedy policy as π, Expected SARSA reduces to **Q-learning** exactly.
- Expected SARSA subsumes both SARSA and Q-learning as special cases.

62 Double Q-Learning

62.1 Maximization Bias

A subtle but important problem arises from the max operator in Q-learning. When action values are estimated with noise (as they always are), selecting the maximum estimated value **introduces a positive bias**:

Maximization Bias

If $Q(s,a) = q(s,a) + \epsilon_a$ where ϵ_a are zero-mean estimation errors, then:

$$\mathbb{E}\left[\max_a Q(s,a)\right] \geq \max_a q(s,a)$$

The maximum of noisy estimates is biased upward. This causes Q-learning to **overestimate** action values, especially in the early stages of learning, and can slow convergence or produce poor policies.

62.2 The Double Q-Learning Solution

The root cause is using the **same** estimates both to *identify* the maximizing action and to *evaluate* its value. Double Q-learning decouples these two operations using two independent value tables:

Double Q-Learning Update

Maintain two action-value functions Q_1 and Q_2. At each step, with probability 0.5:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma\, Q_2\Big(S_{t+1}, \arg\max_a Q_1(S_{t+1}, a)\Big) - Q_1(S_t, A_t)\Big]$$

or symmetrically update Q_2 using Q_1 to evaluate. The action is selected by the sum (or average) of $Q_1 + Q_2$.

- Q_1 is used to **select** the best action: $A^* = \arg\max_a Q_1(S_{t+1}, a)$.
- Q_2 is used to **evaluate** that action: $Q_2(S_{t+1}, A^*)$.
- Because Q_1 and Q_2 are trained on different samples, the selection and evaluation are **unbiased** with respect to each other.
- Double Q-learning converges to q^* and empirically shows dramatically less overestimation than standard Q-learning.

63 n-Step TD Methods

63.1 Between TD and MC

TD(0) uses a one-step return; MC uses the full return. **n-step TD** methods interpolate between these extremes using returns over n steps before bootstrapping:

n-Step Return

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$$

The n-step TD update for prediction is:

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t^{(n)} - V(S_t)]$$

Special cases:

- $n = 1$: reduces to TD(0).
- $n = \infty$ (or $n \geq T - t$): reduces to Monte Carlo.

63.2 Effect of n

Increasing n reduces bias (the bootstrap target is more accurate) but increases variance (more random steps contribute to the return). The optimal n is problem-dependent; empirically intermediate values (e.g., $n = 4$ to $n = 16$) often outperform both TD(0) and MC.

63.3 n-Step SARSA

For control, the n-step return uses action values:

$$G_t^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(S_{t+n}, A_{t+n})$$

and the update rule is:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha[G_t^{(n)} - Q(S_t, A_t)]$$

Remark. n-step methods require storing the last n transitions before making an update. They introduce a delay of n steps before the first update, which is acceptable in practice. $TD(\lambda)$ (covered in later lectures) provides an elegant way to combine all n-step returns with a single eligibility trace mechanism.

64 Summary: TD Learning

64.1 The TD Error as a Unifying Concept

The **TD error** $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is a fundamental concept throughout RL:

- It is the signal that drives all TD updates.
- In neuroscience, dopamine signals in the brain bear remarkable resemblance to TD errors, providing biological support for the TD hypothesis.
- The MC return can be decomposed as a sum of TD errors:

$$G_t - V(S_t) = \sum_{k=t}^{T-1} \gamma^{k-t} \delta_k$$

  This identity links MC and TD and underlies the theory of eligibility traces.

64.2 Algorithm Comparison

Algorithm	On/Off-policy	Target	Key Feature
TD(0)	On-policy	$R + \gamma V(S')$	Prediction only
SARSA	On-policy	$R + \gamma Q(S', A')$	Safe, conservative
Q-learning	Off-policy	$R + \gamma \max_{a'} Q(S', a')$	Optimal policy
Expected SARSA	On or Off	$R + \gamma \mathbb{E}_\pi[Q(S', \cdot)]$	Lower variance
Double Q-learning	Off-policy	$R + \gamma Q_2(S', \arg\max_{a'} Q_1(S', a'))$	No overestimation

Why TD Learning Matters

TD learning is the backbone of most practical RL algorithms, including DQN (Deep Q-Network), which combines Q-learning with deep neural networks and was the first algorithm to achieve human-level performance across a wide range of Atari games. The core ideas — online updates, bootstrapping from the next state's estimate, and the TD error — extend naturally from tabular settings to function approximation.

Looking Ahead

- **n-step TD and TD(λ)**: unify TD(0) and MC with eligibility traces, allowing efficient credit assignment over multiple timesteps.
- **Planning and Learning** (Lecture 7): Dyna architecture — combining TD learning from real experience with simulated experience from a learned model.
- **Function approximation**: scaling TD algorithms to large/continuous state spaces using linear features or neural networks (DQN, Actor-Critic).

Part IX

On-Policy Prediction with Approximation

This part opens the **second part of the course**, which addresses the question: **how can we extend tabular RL methods to apply them to problems with arbitrarily large state spaces?** For instance, the number of possible camera images already exceeds the number of atoms in the universe, so almost all states the agent encounters have never been seen before. The only way forward is **generalization** from previously encountered (similar) states, accepting that optimal policies must be replaced by good *approximate* solutions.

The strategy is to combine **RL with function approximation** (a tool from supervised learning). However, the RL setting introduces **new issues** that supervised learning normally does not face, such as **nonstationarity** of the target, **bootstrapping**, and **delayed targets**.

Goal of this Lecture

Substitute **tabular representations** of the state-value function v_π with **function approximations** $\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$, where $\mathbf{w} \in \mathbb{R}^d$ is a vector of parameters. Approximated values v_π are estimated from **on-policy** data, i.e., from experience generated using the known policy π.

The function $\hat{v}$ may take many forms:

- a **linear function** in features of the state, with $\mathbf{w}$ a vector of feature weights;
- a **multilayer artificial neural network**, with $\mathbf{w}$ the vector of connection weights;
- a **decision tree**, with $\mathbf{w}$ the split points and leaf values.

By adjusting the weights, several functions can be implemented.

Typically the number of weights is much smaller than the number of states, i.e., $d \ll |\mathcal{S}|$. Therefore **changing one weight changes the estimated value of many states**: this is the source of **generalization**. Generalization makes reinforcement learning **more powerful** but also more **difficult to manage and understand**. Extending RL to function approximation also makes it applicable to **partially observable problems**, where the full state is not available.

65 Value-Function Approximation

65.1 Updates as Input-Output Examples

All prediction methods seen so far are based on **updates** of an estimated value function. Each update can be interpreted as an **example** of the desired input-output behavior of the value function:

$$s \longrightarrow \boxed{v_\pi} \longrightarrow v_\pi(s)$$

Different families produce different update targets:

- **MC update**: $S_t \mapsto G_t$, where the target is the observed return.
- **TD update**: $S_t \mapsto R_{t+1} + \gamma\, \hat{v}(S_{t+1}, \mathbf{w}_t)$, where the target is bootstrapped.
- **DP update**: $s \mapsto \mathbb{E}_\pi[R_{t+1} + \gamma\, \hat{v}(S_{t+1}, \mathbf{w}_t) \mid S_t = s]$.

Tabular vs Function-Approximation Updates

- **Tabular representation**: the table entry for the estimated value of state s is shifted a fraction of the way towards the target u. *Estimated values of other states are unchanged.*
- **Function approximation**: arbitrarily complex parameter updates are available. *Updating at state s can change value estimates of other states* (generalization).

Since each update has the form "input state → desired output target", **supervised learning** can be used to compute the weights $\mathbf{w}$ via function approximation methods.

65.2 Why Not Any Supervised Learner?

Not all function approximation methods are equally well suited for RL. In RL, learning must be performed **online**, while the agent interacts with the environment. We need learning methods that:

- learn efficiently from **incrementally acquired data**;
- handle **nonstationary target functions**.

A typical example: in **Generalized Policy Iteration (GPI)** we seek to learn q_π as π itself changes. Methods that cannot deal with such nonstationarity are less suitable for RL.

66 The Prediction Objective

66.1 Mean Squared Value Error

Which objective do we use to evaluate the approximated function? In the tabular case a continuous measure of prediction quality was **not necessary** because:

- the learned value function could become equal to the true one;
- updates affect only single states.

With function approximation these two assumptions are not guaranteed. We must therefore decide **which states we care most about** by defining a **state distribution** $\mu(s) \geq 0$, $\sum_s \mu(s) = 1$.

Mean Squared Value Error

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_\pi(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

The square root of $\overline{\text{VE}}$ provides a **measure** of how much the approximate values differ from the true values.

66.2 The On-Policy Distribution

Often $\mu(s)$ is set to the **fraction of time spent in state** s, the so-called **on-policy distribution**.

In **episodic tasks**, let $h(s)$ be the probability that an episode starts in state s. Then the number of time steps spent, on average, in state s in a single episode is

$$\eta(s) = h(s) + \sum_{\bar{s}} \eta(\bar{s}) \sum_a \pi(a \mid \bar{s})\, p(s \mid \bar{s}, a), \quad \text{for all } s \in \mathcal{S}$$

and the on-policy distribution is the normalization

$$\mu(s) \;=\; \frac{\eta(s)}{\sum_{s'} \eta(s')}, \quad \text{for all } s \in \mathcal{S}.$$

In **continuing tasks** the on-policy distribution is the **stationary distribution** under π. In **episodic tasks** it also depends on the state initial probability. The formal analysis of the continuing and episodic cases must be **treated separately** with value function approximation.

66.3 Global vs Local Optima

The goal of $\overline{\text{VE}}$ is to find a **global optimum**, namely a weight vector $\mathbf{w}^*$ such that

$$\overline{\text{VE}}(\mathbf{w}^*) \;\leq\; \overline{\text{VE}}(\mathbf{w}) \quad \text{for all possible } \mathbf{w}.$$

Local vs Global Optima

A global optimum is **possible** for **simple function approximators** (e.g., linear models), and **rarely** for **complex approximators** (e.g., ANNs and decision trees) in which learning usually converges to **local optima**, i.e., a $\mathbf{w}^*$ such that

$$\overline{\text{VE}}(\mathbf{w}^*) \;\leq\; \overline{\text{VE}}(\mathbf{w}) \quad \text{for all } \mathbf{w} \text{ in some neighborhood of } \mathbf{w}^*.$$

This is the best that can be done and is usually enough, although in many cases there are **no guarantees of convergence to the optimum**.

In Summary

So far we have described:

- a **framework** for combining RL methods for value prediction with function approximation methods (using RL updates as training examples);
- a $\overline{\text{VE}}$ **performance measure** that these methods may aspire to minimize.

In the rest of the lecture we consider function approximation methods based on **gradient-descent** since they are particularly promising and reveal key theoretical properties.

67 Stochastic-Gradient and Semi-Gradient Methods

67.1 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a class of learning methods for function approximation in value prediction. It is among the most widely used of all function approximation methods and is well suited to **online RL**.

Let:

- $\mathbf{w} \doteq (w_1, w_2, \ldots, w_d)^\top$ a weight vector;
- $\hat{v}(s, \mathbf{w})$ a **differentiable** function of $\mathbf{w}$ for all states s.

At each time step $t = 0, 1, 2, 3, \ldots$ we observe a new example $S_t \mapsto v_\pi(S_t)$ and update $\mathbf{w}_t$. States S_t can be **randomly selected** or they can be **successive states** of an interaction with the environment.

The values $v_\pi(S_t)$ are **unknown**, but even if we could observe their true values, learning the approximate function would still be difficult: the approximator has **limited "resolution"**, there is no $\mathbf{w}$ that gets all the states exactly correct.

Goal and Strategy of SGD

Goal of SGD: minimize the error on the observed examples.
Strategy of SGD: adjust $\mathbf{w}$ after each example by a small amount in the direction that would most reduce the error on that example:

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t - \tfrac{1}{2}\alpha\nabla\big[v_\pi(S_t) - \hat{v}(S_t,\mathbf{w}_t)\big]^2 \\ &= \mathbf{w}_t + \alpha\big[v_\pi(S_t) - \hat{v}(S_t,\mathbf{w}_t)\big]\nabla\hat{v}(S_t,\mathbf{w}_t),\end{aligned}$$

where $\alpha > 0$ and $\nabla f(\mathbf{w}) \doteq \big(\frac{\partial f(\mathbf{w})}{\partial w_1}, \frac{\partial f(\mathbf{w})}{\partial w_2}, \ldots, \frac{\partial f(\mathbf{w})}{\partial w_d}\big)^\top$ is the gradient of f.

The **negative gradient** of the example's squared error is the direction in which the error falls most rapidly. SGD is called *stochastic* when the update is done on only a single sample. Over many examples, making small steps, the effect is to minimize $\overline{\text{VE}}$.

Why only "small" steps? If we completely corrected each example in one step then we would **not balance the error** (which cannot be completely removed) on all samples. Convergence results on SGD assume that α **decreases over time** according to standard stochastic-approximation conditions

$$\sum_t \alpha_t = \infty, \qquad \sum_t \alpha_t^2 < \infty.$$

67.2 Approximate Targets and Unbiasedness

In practice the target output observed at time t, $U_t \in \mathbb{R}$, is **not the true value** $v_\pi(S_t)$, but some **random approximation** of it (e.g., a noisy corrupted value of $v_\pi(S_t)$ or a bootstrapping target). We perform an **approximate update** using U_t:

$$\boxed{\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\big[U_t - \hat{v}(S_t,\mathbf{w}_t)\big]\nabla\hat{v}(S_t,\mathbf{w}_t)}$$

Convergence Condition

If U_t is an **unbiased estimate** of the value, i.e.,

$$\mathbb{E}[U_t \mid S_t = s] = v_\pi(S_t),$$

then $\mathbf{w}_t$ is guaranteed to converge to a local optimum.

67.3 Gradient Monte Carlo

The **Monte Carlo target** $U_t \doteq G_t$ is an unbiased estimate of $v_\pi(S_t)$, hence the SGD version of MC state-value prediction **converges**.

Algorithm 20 Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
for each episode **do**
  Generate an episode $S_0, A_0, R_1, S_1, A_1, \ldots, R_T, S_T$ using π
  for each step of episode, $t = 0, 1, \ldots, T-1$ **do**
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[G_t - \hat{v}(S_t, \mathbf{w})\big]\nabla\hat{v}(S_t, \mathbf{w})$
  end for
end for

Notice: MC provides a **non-bootstrapping estimate** of $v_\pi(S_t)$.

67.4 Semi-Gradient Methods

If a **bootstrapping estimate** of $v_\pi(S_t)$ is used as the target U_t (e.g., in TD and DP), then **convergence is not guaranteed**. The reason is that the convergence proof requires the target to be **independent of $\mathbf{w}_t$**, which is not the case when U_t depends on the current weights through $\hat{v}(S_{t+1}, \mathbf{w}_t)$.

Methods working in this condition are called **semi-gradient (bootstrapping) methods**: they take only *part* of the gradient into account (the gradient of $\hat{v}(S_t, \mathbf{w}_t)$, not of the target).

Advantages of Semi-Gradient Methods

They do not converge as robustly as full gradient methods, but they **converge reliably in important cases** (e.g., the linear case). Their advantages are:

- they enable **faster learning**;
- they enable **continual and online learning**, without waiting for the end of the episode.

Algorithm 21 Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: the policy π to be evaluated
Input: a differentiable function $\hat{v} : \mathcal{S}^+ \times \mathbb{R}^d \to \mathbb{R}$ such that $\hat{v}(\text{terminal}, \cdot) = 0$
Algorithm parameter: step size $\alpha > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
for each episode **do**
  Initialize S
  repeat
    Choose $A \sim \pi(\cdot \mid S)$
    Take action A, observe R, S'
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\,\hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})\big]\nabla\hat{v}(S, \mathbf{w})$
    $S \leftarrow S'$
  until S is terminal
end for

67.5 Example: State Aggregation on the 1000-State Random Walk

State aggregation is a simple generalizing function approximation:

- states are **grouped together** with one estimated value (constant) per group;

- each component of $\mathbf{w}$ is the estimation for a group of states;
- the gradient $\nabla\hat{v}(S_t, \mathbf{w}_t)$ is 1 for the components of the group of S_t and 0 for the other components.

Consider a **1000-state version of the random walk task**: states are arranged on a line with two terminal states; transitions go left or right; the reward is 0 everywhere except +1 on reaching the right terminal state. Function approximation by state aggregation using the **gradient MC algorithm** with 100 000 episodes, $\alpha = 2 \times 10^{-5}$, and 10 groups produces a piecewise-constant approximation $\hat{v}$ that closely matches the true value v_π across the state space, with the on-policy distribution μ peaked in the middle of the chain.

68 Linear Value Function Approximation

68.1 Linear Methods

We approximate $\hat{v}(\cdot, \mathbf{w})$ with a **linear function** of the weight vector $\mathbf{w}$. For each state s there is a real-valued **feature vector**

$$\mathbf{x}(s) \doteq (x_1(s), x_2(s), \ldots, x_d(s))^\top$$

with the same number of components (**features**) as $\mathbf{w}$, i.e., d. The value of each feature is a function of the state, $x_i : \mathcal{S} \to \mathbb{R}$.

Linear State-Value Approximation

Linear-method approximations of the state-value function implement the **inner product** between $\mathbf{w}$ and $\mathbf{x}(s)$:

$$\hat{v}(s, \mathbf{w}) \doteq \mathbf{w}^\top \mathbf{x}(s) \doteq \sum_{i=1}^{d} w_i \, x_i(s).$$

The approximate value-function is said to be **linear in the weights**.

68.2 SGD with Linear Functions

It is natural to use SGD updates with linear function approximations. The gradient of the approximate value function w.r.t. $\mathbf{w}$ in this case is

$$\nabla\hat{v}(s, \mathbf{w}) = \mathbf{x}(s).$$

Hence, the **SGD update** becomes

$$\boxed{\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\big[U_t - \hat{v}(S_t, \mathbf{w}_t)\big]\mathbf{x}(S_t)}$$

The simple form is good for **mathematical analysis** (e.g., convergence) and there is **only one optimum**, so *local optimum = global optimum*.

68.3 Convergence Results in the Linear Case

- The **Gradient Monte Carlo** algorithm **converges** to the global optimum of $\overline{\text{VE}}$ under linear function approximation.
- The **semi-gradient TD(0)** algorithm also **converges** under linear function approximation. This result requires a separate theorem: the weight vector converges to a point **near the local optimum** (the *TD fixed point*).

The update at each time t for linear semi-gradient TD(0) is

$$\begin{aligned}\mathbf{w}_{t+1} &\doteq \mathbf{w}_t + \alpha(R_{t+1} + \gamma\,\mathbf{w}_t^\top \mathbf{x}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t)\mathbf{x}_t \\ &= \mathbf{w}_t + \alpha(R_{t+1}\,\mathbf{x}_t - \mathbf{x}_t\,(\mathbf{x}_t - \gamma\,\mathbf{x}_{t+1})^\top \mathbf{w}_t).\end{aligned}$$

At steady state the expected next weight vector is

$$\mathbb{E}[\mathbf{w}_{t+1} \mid \mathbf{w}_t] \;=\; \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t),$$

with

$$\mathbf{b} \doteq \mathbb{E}[R_{t+1}\,\mathbf{x}_t] \in \mathbb{R}^d, \qquad \mathbf{A} \doteq \mathbb{E}\Big[\mathbf{x}_t\,(\mathbf{x}_t - \gamma\,\mathbf{x}_{t+1})^\top\Big] \in \mathbb{R}^d \times \mathbb{R}^d.$$

68.4 TD Fixed Point and Error Bound

The **TD fixed point** for linear semi-gradient TD(0) is computed as

$$\mathbf{b} - \mathbf{A}\mathbf{w}_{\text{TD}} = \mathbf{0} \;\Rightarrow\; \mathbf{b} = \mathbf{A}\mathbf{w}_{\text{TD}} \;\Rightarrow\; \mathbf{w}_{\text{TD}} \doteq \mathbf{A}^{-1}\mathbf{b}.$$

Bound on the TD Fixed-Point Error

At the TD fixed point it has been proved (in the continuing case) that the $\overline{\text{VE}}$ is within a **bounded expansion** of the lowest possible error:

$$\overline{\text{VE}}(\mathbf{w}_{\text{TD}}) \;\leq\; \frac{1}{1-\gamma} \min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w}).$$

The asymptotic error of the TD method is no more than $\frac{1}{1-\gamma}$ times the smallest possible error, i.e., the error reached in the limit by the Monte Carlo method.

Since γ is usually close to 1, the bound represents a **substantial potential loss**, but in practice TD methods have **reduced variance** and are **faster than MC methods**.

69 Feature Construction for Linear Methods (Hints)

Advantages of linear approximation:

- **convergence guarantees**;
- **data efficiency**;
- **computational efficiency**.

These advantages **depend a lot on how the states are represented** in terms of **features**. Choosing appropriate features requires **prior domain knowledge**.

Choosing Features

Features should correspond to the **aspects of the state space along which generalization may be appropriate**.

- E.g., states of geometric objects: features for each possible shape, color, size, etc.
- E.g., states of a mobile robot: features for location, remaining battery, etc.

Limitation of Linear Approximation

Linear approximation **cannot consider interactions between features**.
E.g., in the **pole-balancing task**, a high angular velocity can be either good or bad depending on the angle. A linear value function cannot represent this if the angle and the angular velocity are coded as separate features.

There exist **several ways to construct meaningful features** (e.g., polynomials, Fourier basis, etc.). This is beyond the scope of the course (see Sec. 9.5 of the Sutton and Barto book for details).

70 Nonlinear Value Function Approximation (Hints)

There exist several **non-linear methods for approximating the value function**, such as:

- **Artificial Neural Networks (ANNs)**;
- **Memory-based (nonparametric) functions**;
- **Kernel-based functions**.

Artificial Neural Networks

ANNs have recently become the most popular approximation functions:
- they are **universal function approximators**;
- in **deep architectures** they can generate **hierarchical representations of features** automatically (vs hand-crafted features);
- they typically learn by **stochastic gradient** methods;
- they can learn value functions (see **Deep Q-Networks** in next slides).

Part X

On-Policy Control with Approximation and Deep Q-Networks

This part addresses the **control problem with function approximation**: instead of approximating the state-value function v_π, we approximate the **action-value function**

$$\hat{q}(s, a, \mathbf{w}) \approx q_*(s, a),$$

where $\mathbf{w} \in \mathbb{R}^d$ is a finite-dimensional weight vector. Attention is first restricted to the **on-policy** and **episodic** case, where the natural extension of semi-gradient TD(0) for prediction becomes the **semi-gradient Sarsa** algorithm for control. The same machinery, combined with deep neural networks and Q-learning, leads to **Deep Q-Networks (DQN)**.

Goal of this Lecture

Extend semi-gradient methods from **value prediction** to **control**:

- move from state-value approximation $\hat{v}(s, \mathbf{w})$ to action-value approximation $\hat{q}(s, a, \mathbf{w})$;
- move from **prediction** to **on-policy control** via ε-greedy improvement;
- introduce **deep neural networks** as nonlinear approximators, yielding the **DQN** algorithm of Mnih et al. (2015).

71 Episodic Semi-Gradient Control

71.1 From State-Value to Action-Value Approximation

The extension of state-value function approximators $\hat{v}(s, \mathbf{w})$ to **action-value function approximators** $\hat{q}(s, a, \mathbf{w})$ is **straightforward**. The only structural change is in the form of training examples:

- **State-value functions:** training examples of the form $S_t \mapsto U_t$;
- **Action-value functions:** training examples of the form $S_t, A_t \mapsto U_t$.

The **update target** U_t can be any approximation of $q_\pi(S_t, A_t)$, including:

- the **full Monte Carlo return** G_t;
- the **Sarsa return** $R_{t+1} + \gamma\, \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t)$.

71.2 The General Gradient-Descent Update

The **general gradient-descent update** for action-value prediction is

$$\boxed{\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\big[U_t - \hat{q}(S_t, A_t, \mathbf{w}_t)\big]\nabla \hat{q}(S_t, A_t, \mathbf{w}_t)}$$

For the **one-step Sarsa** method (bootstrapped target) it becomes

$$\boxed{\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\big[R_{t+1} + \gamma\, \hat{q}(S_{t+1}, A_{t+1}, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t)\big]\nabla \hat{q}(S_t, A_t, \mathbf{w}_t)}$$

This method is called **episodic semi-gradient one-step Sarsa**.

71.3 Policy Improvement and Action Selection

Control methods need to couple two ingredients:

- methods for **action-value prediction**;
- methods for **policy improvement and action selection**.

If the **action set is discrete and not too large**, the techniques from tabular methods carry over directly.

Idea: Greedy Action Selection with Approximation

- For each action a of the current state S_t, compute $\hat{q}(S_t, a, \mathbf{w}_t)$.
- Find the **greedy action**

$$A_t^* \;=\; \arg\max_a \hat{q}(S_t, a, \mathbf{w}_t).$$

- Perform **policy improvement** by changing the estimation policy to a **soft approximation** of the greedy policy (e.g., ε-greedy).

71.4 Algorithm: Episodic Semi-Gradient Sarsa

Algorithm 22 Episodic Semi-Gradient Sarsa for Estimating $\hat{q} \approx q_*$

Input: a differentiable action-value function parameterization $\hat{q} : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$
Algorithm parameters: step size $\alpha > 0$, small $\varepsilon > 0$
Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g., $\mathbf{w} = \mathbf{0}$)
for each episode **do**
  $S, A \leftarrow$ initial state and action of episode (e.g., ε-greedy)
  for each step of episode **do**
    Take action A, observe R, S'
    if S' is terminal **then**
      $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
      Go to next episode
    end if
    Choose A' as a function of $\hat{q}(S', \cdot, \mathbf{w})$ (e.g., ε-greedy)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha\big[R + \gamma\,\hat{q}(S', A', \mathbf{w}) - \hat{q}(S, A, \mathbf{w})\big]\nabla\hat{q}(S, A, \mathbf{w})$
    $S \leftarrow S'$, $A \leftarrow A'$
  end for
end for

71.5 Example: Mountain Car

A canonical benchmark for semi-gradient Sarsa with linear function approximation is the **Mountain Car** task.

Setting:

- **Actions:** full throttle forward ($+1$), full throttle reverse (-1), zero throttle (0);
- **Reward:** -1 at each step until the car reaches the goal and the episode terminates.

Simplified physics:

$$\begin{aligned} x_{t+1} &\doteq \text{bound}\big[x_t + \dot{x}_{t+1}\big], \\ \dot{x}_{t+1} &\doteq \text{bound}\big[\dot{x}_t + 0.001A_t - 0.0025\cos(3x_t)\big], \end{aligned}$$

with bounds $-1.2 \le x_{t+1} \le 0.5$ and $-0.07 \le \dot{x}_{t+1} \le 0.07$. Episodes start in a random position $x_t \in [-0.6, -0.4)$ with zero velocity.

Feature representation. The two continuous state variables are converted to **binary features** via **grid tiling**: 8 tilings, each tile covering 1/8-th of the bounded distance in each dimension, with asymmetric offsets (see Sec. 9.5.4 of Sutton & Barto). The feature vectors are combined

linearly to approximate the action-value function

$$\hat{q}(s, a, \mathbf{w}) \doteq \mathbf{w}^\top \mathbf{x}(s, a) = \sum_{i=1}^{d} w_i \, x_i(s, a)$$

for each pair (s, a).

Observations. Starting from all-zero (optimistic) initial action values, the agent performs **extensive exploration** even under a null policy. The cost-to-go surface $(-\max_a \hat{q}(s, a, \mathbf{w}))$ shapes progressively over episodes. Learning curves for semi-gradient Sarsa with tile coding and ε-greedy action selection show **faster convergence for larger step sizes** (e.g., $\alpha = 0.5/8$ vs $\alpha = 0.1/8$), measured in steps per episode (log scale, averaged over 100 runs).

72 Deep Q-Networks (DQN)

72.1 ANNs for Value Function Approximation in RL

Multi-layer ANNs have been used for function approximation in RL since **1986**, when the **backpropagation** algorithm became popular as a method for learning internal representations (Rumelhart et al., 1986).

Striking results were obtained by coupling RL with backpropagation in **TD-Gammon** and **Watson** (Tesauro et al., 1994; 2012).

In **2013**, Mnih and colleagues (Google DeepMind) developed the first RL agent merging **Q-learning** with **deep convolutional ANNs**, called **Deep Q-Network (DQN)**, achieving **human-level performance** on **Atari** games. Like TD-Gammon, DQN uses a **semi-gradient** form of a TD algorithm with gradients computed by backpropagation, but it employs **Q-learning** instead of TD(λ).

72.2 Architecture and Update Rule

Basic Idea of DQN

Use a **deep neural network** as a **non-linear function approximator** for the action-value function in a **semi-gradient form of Q-learning**.

The approximate value function $\hat{q}(s, a, \mathbf{w}_t)$ is parametrized by a **deep convolutional neural network**, with $\mathbf{w}_t$ the weights at iteration t. The network is called the **Q-network**.

- **Input of the Q-network:** raw sensor signals (current state). Deep NNs can perform **automatic feature construction**, generating meaningful hierarchical abstractions in their layers.
- **Output of the Q-network:** estimated optimal action values for the input state, i.e. **one value per action**.

The **semi-gradient Q-learning update** used by DQN is

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[\underbrace{R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)}_{\text{target value}} - \underbrace{\hat{q}(S_t, A_t, \mathbf{w}_t)}_{\text{action value}} \Big] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

where $\mathbf{w}_t$ is the vector of network weights, A_t is the action selected at step t, and S_t, S_{t+1} are the network inputs. The gradient $\nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$ is computed via **backpropagation**.

72.3 Problems with Nonlinear Approximation

Instability of Nonlinear RL

Problem: RL is **unstable** or even **diverges** with nonlinear function approximators (e.g. ANNs) of the action-value function (Mnih et al., 2015).

Causes of instability:

- **C1: correlations** in the sequences of observations (consecutive states/features are highly correlated);
- **C2:** small updates to $\hat{q}$ may **significantly change the policy** and thus the data distribution;
- **C3: correlation between action values** $\hat{q}(S_t, A_t, \mathbf{w}_t)$ and **target values** $R_{t+1} + \gamma \max_a \hat{q}(S_{t+1}, a, \mathbf{w}_t)$, since both depend on the same weights $\mathbf{w}_t$.

Two key solutions were proposed in (Mnih et al., 2015):

1. A biologically inspired mechanism for **experience replay**;
2. The usage of **two separate networks** to estimate action values and target values.

72.4 Experience Replay

Idea. Store agent experience in a **replay memory**, then use stored experience to perform weight updates. After each step a tuple $(S_t, A_t, R_{t+1}, S_{t+1})$ is added to the replay memory. Experience is accumulated across many episodes.

At each step, **multiple Q-learning updates** (a **mini-batch**) are performed using experience **sampled uniformly at random** from the replay memory. Notice: since Q-learning is **off-policy**, it can be applied along unconnected trajectories.

Advantages of Experience Replay

- **Reduced variance** of the weight updates (mitigates cause **C2**);
- The **correlation in the sequences of observations** is eliminated → one source of instability is removed (mitigates cause **C1**).

72.5 Two Networks (Target Network)

Two networks are used: one for estimating **action values** (ANN 1, the Q-network), another for estimating **target values** (ANN 2, the target network with parameters $\tilde{\mathbf{w}}$). The update rule becomes

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[R_{t+1} + \gamma \max_a \tilde{q}(S_{t+1}, a, \tilde{\mathbf{w}}) - \hat{q}(S_t, A_t, \mathbf{w}_t) \Big] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t).$$

After every C updates of the weights $\mathbf{w}$ of the action-value network (ANN 1), these weights are **copied** to the second network (ANN 2) used to compute the target values.

Advantage of the Target Network

This decoupling **improves stability** by reducing the correlation between action values and target values (mitigates cause **C3**).

72.6 Experimental Setting (Mnih et al., 2015)

Benchmark. DQN was first evaluated on **49 Atari games** (Mnih et al., 2013; 2015).

Input pipeline:

- raw frames of 210×160 pixels, 128 colors, 60Hz;
- **preprocessing:** images reduced to 84×84 arrays of luminance;
- **stacked images:** the four most recent frames are provided at each step, so the actual input has dimension $84 \times 84 \times 4$.

Network architecture:

- 3 hidden **convolutional layers** with rectifier nonlinearities:
  - 32 feature maps of size 20×20,
  - 64 feature maps of size 9×9,
  - 64 feature maps of size 7×7;
- 1 fully connected hidden layer with 512 neurons;
- output layer with 18 neurons (one per Atari action).

Reward shaping: $+1$ for increased game score, -1 for decreased score, 0 otherwise.

Policy: ε-greedy with ε decreasing linearly over the first million frames, low value afterwards (50M frames in total, i.e. $\sim$ 38 days of game time).

Hyperparameters: input, output, ANN architecture, step size, discount factor, etc., were tuned on a small selection of games and then kept **fixed for all games** (a test of generalization). Learning was performed **independently for each game** (different weights per game).

72.7 Results

Evaluations were performed on 30 sessions of each game, each lasting up to 5 minutes and beginning in a random initial state. According to Mnih et al. (2015), DQN performed:

- **better than the state-of-the-art** algorithm (linear function approximation with hand-crafted features, Bellemare et al., 2013) in **43 games**;
- at a level **comparable to professional human players** in **29 games**.

Take-Away

DQN demonstrates that **deep nonlinear approximators** can be combined with semi-gradient **Q-learning** to produce a general-purpose RL agent operating directly on **raw pixel inputs**, provided that the two stabilization tricks — **experience replay** and a **target network** — are used. These two ideas address the instability causes **C1, C2, C3** listed above and form the backbone of modern value-based deep RL.

Part XI

Policy Gradient Methods

All methods seen so far are **action-value methods**: they **estimate** action values, **select** actions based on these values, but **do not explicitly represent the policy function**. **Policy gradient methods** take a different route: they learn a **parametrized policy function** and select actions using this policy *without consulting value functions*. A value function may still be learned, but only **to help update the policy parameters**.

Actor-critic methods are policy gradient methods that *also* learn an approximation of the value function:

- **Actor:** the learned policy;
- **Critic:** the learned value function (used for bootstrapping).

Notation

- $\boldsymbol{\theta} \in \mathbb{R}^{d'}$: **policy parameter vector**;
- $\pi(a \mid s, \boldsymbol{\theta}) = \Pr\{A_t = a \mid S_t = s, \boldsymbol{\theta}_t = \boldsymbol{\theta}\}$: probability of taking action a at time t in state s under parameters $\boldsymbol{\theta}$;
- $\hat{v}(s, \mathbf{w})$: learned value function, with parameters $\mathbf{w} \in \mathbb{R}^d$ (if required by the method);
- $J(\boldsymbol{\theta})$: **performance measure** of the policy as a function of $\boldsymbol{\theta}$.

Goal of Policy Gradient Methods

Learn parameters $\boldsymbol{\theta}$ that **maximize** $J(\boldsymbol{\theta})$, via approximate **gradient ascent**:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha \widehat{\nabla J(\boldsymbol{\theta}_t)},$$

where $\widehat{\nabla J(\boldsymbol{\theta}_t)} \in \mathbb{R}^{d'}$ is a **stochastic estimate** of the gradient of $J(\boldsymbol{\theta})$ w.r.t. $\boldsymbol{\theta}_t$.

The performance measure has two standard forms:

- **Episodic case:** $J(\boldsymbol{\theta}) \doteq v_{\pi_{\boldsymbol{\theta}}}(s_0)$, the value of the start state under the parametrized policy;
- **Continuing case:** $J(\boldsymbol{\theta})$ is the **average reward rate**.

73 Policy Approximation and its Advantages

73.1 Parametrization Requirements

The policy can be **parametrized in any way**, provided $\pi(a \mid s, \boldsymbol{\theta})$ is **differentiable** w.r.t. its parameters. To ensure **exploration**, the policy must **never become deterministic**, i.e. $\pi(a \mid s, \boldsymbol{\theta}) \in (0, 1)$ for all (s, a).

73.2 Soft-max in Action Preferences

For a **discrete (and not too large) action space**, a natural parametrization uses **numerical preferences** $h(s, a, \boldsymbol{\theta}) \in \mathbb{R}$ for each state-action pair. Probability is assigned to actions

proportionally to preferences through an **exponential soft-max**:

$$\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{e^{h(s,a,\boldsymbol{\theta})}}{\sum_b e^{h(s,b,\boldsymbol{\theta})}}$$

This is called **soft-max in action preferences**.

The preferences themselves can be parametrized arbitrarily:

- a **deep ANN** computing preferences (e.g. as in AlphaGo), where $\boldsymbol{\theta}$ is the vector of connection weights;
- a **linear-in-features** preference $h(s, a, \boldsymbol{\theta}) = \boldsymbol{\theta}^\top \mathbf{x}(s, a)$, with $\mathbf{x}(s, a) \in \mathbb{R}^{d'}$ a feature vector of the policy.

Preferences vs. Action Values

Action values *could* be used as preferences with a soft-max, but this would **not allow the policy to approach deterministic behaviour** (action values converge to specific scalars). Instead, **general action preferences** are not constrained to converge to any particular value: preferences of **optimal actions** can be driven **infinitely higher** than those of suboptimal ones, enabling deterministic limit policies.

73.3 Advantages of Policy Parametrization

Why Parametrize the Policy?

- The **policy** may be a **simpler** function to approximate than the value function;
- Policy parametrization **allows to inject prior knowledge** — often the most important reason for using a policy-based method;
- Action-value methods have **no natural way of finding stochastic policies**; policy gradient methods (e.g. with soft-max in action preferences) enable arbitrary action probabilities;
- Policy-based methods can deal with **continuous action spaces**.

73.4 Example: Short Corridor with Switched Actions

Consider a **short corridor** with three internal states and one terminal goal, reward -1 per step, actions $\{\text{left}, \text{right}\}$, and the **middle state** secretly reversing the effect of actions. State features are

$$\mathbf{x}(s, \text{right}) = [1, 0]^\top, \qquad \mathbf{x}(s, \text{left}) = [0, 1]^\top \quad \text{for all states } s.$$

Under **function approximation** all states look identical (model-free).

Action-value method (ε-greedy): since all states share the same features, only two extreme policies are possible — pick *right* with probability $(1 - \varepsilon)/2 + \varepsilon/2$ or *left* with that probability. Both perform badly ($J \approx -82$ for left-greedy, $J \approx -44$ for right-greedy).

Policy gradient method: can converge to the **optimal stochastic policy**, with the *best probability of right* ≈ 0.59 and performance $J \approx -11.6$.

74 The Policy Gradient Theorem

74.1 Motivation

With continuous policy parametrization, the action probabilities **change smoothly** as a function of $\boldsymbol{\theta}$, instead of changing **dramatically** as in ε-greedy on small changes of action values. Because of this, **stronger convergence guarantees** are available for policy gradient methods.

Performance depends on both **action selection** and the **distribution of states**, both affected by $\boldsymbol{\theta}$. Given a state, the effect of $\boldsymbol{\theta}$ on actions and rewards can be computed, but the effect of the policy on the **state distribution** is a function of the **unknown environment** (model-free setting).

Central Question

How can we estimate the **performance gradient** w.r.t. the policy parameters when the gradient depends on the **unknown effect of policy changes on the state distribution**?

The **Policy Gradient Theorem** answers this question with an analytic expression for the gradient that **does not involve the derivative of the state distribution**.

74.2 Statement of the Theorem (Episodic Case)

Policy Gradient Theorem (episodic case)

$$\nabla J(\boldsymbol{\theta}) \;\propto\; \sum_{s} \mu(s) \sum_{a} q_\pi(s,a)\, \nabla \pi(a \mid s, \boldsymbol{\theta})$$

where gradients are column vectors of partial derivatives w.r.t. components of $\boldsymbol{\theta}$, and $\mu(s)$ is the **on-policy state distribution** under π (parametrized by $\boldsymbol{\theta}$).

The **constant of proportionality** is the **average length of an episode** in the episodic case and 1 in the continuing case.

74.3 Proof Sketch

Starting from

$$\nabla v_\pi(s) \;=\; \nabla \Big[\sum_{a} \pi(a \mid s)\, q_\pi(s,a) \Big]$$

and applying the product rule,

$$\begin{aligned} \nabla v_\pi(s) \;&=\; \sum_{a} \Big[\nabla \pi(a \mid s)\, q_\pi(s,a) + \pi(a \mid s)\, \nabla q_\pi(s,a) \Big] \\ &=\; \sum_{a} \Big[\nabla \pi(a \mid s)\, q_\pi(s,a) + \pi(a \mid s) \sum_{s'} p(s' \mid s,a)\, \nabla v_\pi(s') \Big]. \end{aligned}$$

Unrolling recursively yields

$$\nabla v_\pi(s) \;=\; \sum_{x \in \mathcal{S}} \sum_{k=0}^{\infty} \Pr(s \to x, k, \pi) \sum_{a} \nabla \pi(a \mid x)\, q_\pi(x,a),$$

where $\Pr(s \to x, k, \pi)$ is the probability of transitioning from s to x in k steps under π. Setting $s = s_0$ and normalizing via $\mu(s) \propto \eta(s) = \sum_k \Pr(s_0 \to s, k, \pi)$ produces

$$\nabla J(\boldsymbol{\theta}) \;=\; \nabla v_\pi(s_0) \;\propto\; \sum_{s} \mu(s) \sum_{a} q_\pi(s,a)\, \nabla \pi(a \mid s, \boldsymbol{\theta}). \qquad \square$$

(See Sutton & Barto p. 325 for the full derivation.)

75 REINFORCE: Monte-Carlo Policy Gradient

75.1 All-Actions Update

Stochastic gradient ascent requires **samples** whose expectation is proportional to $\nabla J(\boldsymbol{\theta}_t)$. The policy gradient theorem provides exactly that:

$$\nabla J(\boldsymbol{\theta}) \;\propto\; \sum_s \mu(s) \sum_a q_\pi(s,a)\, \nabla \pi(a \mid s, \boldsymbol{\theta}) \;=\; \mathbb{E}_\pi\left[\sum_a q_\pi(S_t,a)\, \nabla \pi(a \mid S_t, \boldsymbol{\theta})\right],$$

since under π states are encountered according to $\mu(s)$. A first algorithm — called **all-actions** because the update sums over all actions — is

$$\boldsymbol{\theta}_{t+1} \;\doteq\; \boldsymbol{\theta}_t + \alpha \sum_a \hat{q}(S_t, a, \mathbf{w})\, \nabla \pi(a \mid S_t, \boldsymbol{\theta}),$$

with $\hat{q}$ a learned approximation of q_π.

75.2 Derivation of REINFORCE

Considering only the action A_t actually taken at time t yields the **REINFORCE algorithm**. Multiplying and dividing the summed terms by $\pi(a \mid S_t, \boldsymbol{\theta})$:

$$\begin{aligned}
\nabla J(\boldsymbol{\theta}) \;&=\; \mathbb{E}_\pi\left[\sum_a \pi(a \mid S_t, \boldsymbol{\theta})\, q_\pi(S_t,a)\, \frac{\nabla \pi(a \mid S_t, \boldsymbol{\theta})}{\pi(a \mid S_t, \boldsymbol{\theta})}\right] \\
&=\; \mathbb{E}_\pi\left[q_\pi(S_t, A_t)\, \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta})}{\pi(A_t \mid S_t, \boldsymbol{\theta})}\right] \quad \text{(replacing } a \text{ by sample } A_t \sim \pi) \\
&=\; \mathbb{E}_\pi\left[G_t\, \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta})}{\pi(A_t \mid S_t, \boldsymbol{\theta})}\right] \quad \text{(since } \mathbb{E}_\pi[G_t \mid S_t, A_t] = q_\pi(S_t, A_t)),
\end{aligned}$$

where G_t is the return. This is exactly what we need: **a quantity that can be sampled on each time step, whose expectation equals the gradient**.

75.3 REINFORCE Update Rule

REINFORCE Update Rule

$$\boldsymbol{\theta}_{t+1} \;\doteq\; \boldsymbol{\theta}_t + \alpha\, G_t\, \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)}{\pi(A_t \mid S_t, \boldsymbol{\theta}_t)}.$$

Interpretation: each increment is the product of the return G_t and the vector $\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)/\pi(A_t \mid S_t, \boldsymbol{\theta}_t)$.

- $\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)$ is the **direction in parameter space** that most increases the probability of repeating A_t on future visits to S_t;
- the update moves $\boldsymbol{\theta}$ in this direction **proportionally to the return** (favouring directions yielding highest returns) and **inversely proportionally to the action probability** (removing the advantage of frequently-selected actions that may not be the best).

Using $\nabla \ln x = \nabla x / x$, the update is equivalently

$$\boldsymbol{\theta}_{t+1} \;\doteq\; \boldsymbol{\theta}_t + \alpha\, G_t\, \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta}_t).$$

75.4 Algorithm (Williams, 1992)

Algorithm 23 REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a \mid s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g. $\boldsymbol{\theta} = \mathbf{0}$)
for each episode **do**
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot \mid \cdot, \boldsymbol{\theta})$
  for each step $t = 0, 1, \ldots, T-1$ **do**
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k \quad (= G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha\, \gamma^t\, G\, \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta})$
  end for
end for

75.5 Properties

Theoretical Properties

- REINFORCE uses the **complete return** G_t from time t (all rewards until the end of the episode), so it is a **Monte-Carlo algorithm**, well defined for the **episodic** case;
- The **expected update over an episode** is in the same direction as the performance gradient;
- This assures **improvement of expected performance** for sufficiently small α and **convergence to a local optimum** under standard stochastic-approximation conditions on decreasing α;
- As a Monte-Carlo method, REINFORCE may suffer from **high variance** and consequently **slow learning**.

Empirically on the short corridor task, REINFORCE converges close to the optimal stochastic policy value $v_*(s_0) \approx -11.6$ over ~ 1000 episodes, with the step size α governing speed and final accuracy (e.g. $\alpha = 2^{-13}$ converges fastest among the tested values).

76 REINFORCE with Baseline

76.1 Generalized Policy Gradient Theorem

The policy gradient theorem can be **generalized** by subtracting an arbitrary **baseline** $b(s)$ from the action value:

$$\nabla J(\boldsymbol{\theta}) \;\propto\; \sum_s \mu(s) \sum_a \Big(q_\pi(s,a) - b(s) \Big) \nabla \pi(a \mid s, \boldsymbol{\theta}).$$

The equation remains valid as long as $b(s)$ **does not depend on** a, because the subtracted term integrates to zero:

$$\sum_a b(s)\, \nabla \pi(a \mid s, \boldsymbol{\theta}) \;=\; b(s)\, \nabla \sum_a \pi(a \mid s, \boldsymbol{\theta}) \;=\; b(s)\, \nabla 1 \;=\; 0.$$

76.2 Update Rule with Baseline

$$\boxed{\boldsymbol{\theta}_{t+1} \;\doteq\; \boldsymbol{\theta}_t + \alpha \left(G_t - b(S_t) \right) \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)}{\pi(A_t \mid S_t, \boldsymbol{\theta}_t)}}$$

The baseline:

- **leaves the expected value of the update unchanged** (unbiased);
- can have a **positive effect on variance**;
- **can vary with the state**: in states where all actions have high values, a high baseline helps differentiate good from bad actions; in low-value states, a low baseline is appropriate.

A **natural choice** is an estimate of the state value $\hat{v}(S_t, \mathbf{w})$, with $\mathbf{w}$ learned by value-based methods.

76.3 Algorithm

Algorithm 24 REINFORCE with Baseline (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a \mid s, \boldsymbol{\theta})$
Input: a differentiable state-value parameterization $\hat{v}(s, \mathbf{w})$
Algorithm parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and $\mathbf{w} \in \mathbb{R}^d$ (e.g. to $\mathbf{0}$)
for each episode **do**
  Generate $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$ following $\pi(\cdot \mid \cdot, \boldsymbol{\theta})$
  for each step $t = 0, 1, \ldots, T-1$ **do**
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$
    $\delta \leftarrow G - \hat{v}(S_t, \mathbf{w})$
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}} \, \delta \, \nabla \hat{v}(S_t, \mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \, \gamma^t \, \delta \, \nabla \ln \pi(A_t \mid S_t, \boldsymbol{\theta})$
  end for
end for

Empirically, REINFORCE with baseline **converges faster** and reaches a value closer to $v_*(s_0)$ than vanilla REINFORCE (e.g. on the short-corridor task, $\alpha^{\boldsymbol{\theta}} = 2^{-9}$, $\alpha^{\mathbf{w}} = 2^{-6}$ reach near-optimum within ~ 200 episodes).

77 Actor-Critic Methods

77.1 Why Not Actor-Critic Already?

REINFORCE-with-baseline learns both **a policy** and **a state-value function**, but we do **not** consider it an actor-critic method, because the state-value function is **not used as a critic**: it is **not used for bootstrapping** (updating the value estimate of a state from estimates of subsequent states). It is used only as a baseline for the state whose estimate is being updated.

77.2 Bootstrapping Critic

Bootstrapping bias is often **beneficial**: it reduces variance and accelerates learning. REINFORCE with baseline is unbiased and converges to a local minimum, but **Monte-Carlo methods learn slowly** and are inconvenient online for continuing problems.

Temporal-Difference methods eliminate these inconveniences. Combining TD with policy gradients yields **Actor-Critic methods with a bootstrapping critic** — a **TD version of policy gradient**.

77.3 One-Step Actor-Critic Update

One-step actor-critic methods replace the full return of REINFORCE with the **one-step return** $G_{t:t+1}$ and use a learned $\hat{v}$ as the baseline:

$$\begin{aligned}\boldsymbol{\theta}_{t+1} &\doteq \boldsymbol{\theta}_t + \alpha \left(G_{t:t+1} - \hat{v}(S_t, \mathbf{w})\right) \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)}{\pi(A_t \mid S_t, \boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha \left(R_{t+1} + \gamma\, \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})\right) \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)}{\pi(A_t \mid S_t, \boldsymbol{\theta}_t)} \\ &= \boldsymbol{\theta}_t + \alpha\, \delta_t \frac{\nabla \pi(A_t \mid S_t, \boldsymbol{\theta}_t)}{\pi(A_t \mid S_t, \boldsymbol{\theta}_t)}.\end{aligned}$$

Here $\hat{v}(S_{t+1}, \mathbf{w})$ is used for **bootstrapping** — this is what makes $\hat{v}$ the **critic**.

The natural way to learn $\hat{v}$ in this context is **semi-gradient TD(0)**.

77.4 Roles: Actor, Critic, Advantage

Components of Actor-Critic

- $\pi(A_t \mid S_t, \boldsymbol{\theta}_t)$: **actor**;
- $\hat{v}(S_t, \mathbf{w})$: **critic**;
- $R_{t+1} + \gamma\, \hat{v}(S_{t+1}, \mathbf{w})$: **target**;
- $\delta_t = R_{t+1} + \gamma\, \hat{v}(S_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})$: **advantage** (TD error).

The **advantage** tells us whether an action is better or worse than expected:

- **Advantage** > 0: action is better than expected $\rightarrow$ encourage it;
- **Advantage** < 0: action is worse than expected $\rightarrow$ discourage it (encourage the opposite).

77.5 Algorithm: Advantage Actor-Critic (A2C)

Algorithm 25 One-step Actor-Critic (episodic), for estimating $\pi_{\boldsymbol{\theta}} \approx \pi_*$

Input: a differentiable policy parameterization $\pi(a \mid s, \boldsymbol{\theta})$
Input: a differentiable state-value parameterization $\hat{v}(s, \mathbf{w})$
Parameters: step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\mathbf{w}} > 0$
Initialize $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ and $\mathbf{w} \in \mathbb{R}^{d}$ (e.g. to $\mathbf{0}$)
for each episode **do**
  Initialize S (first state of episode)
  $I \leftarrow 1$
  while S is not terminal **do**
    $A \sim \pi(\cdot \mid S, \boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma\, \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$ (if S' is terminal, $\hat{v}(S', \mathbf{w}) \doteq 0$)
    $\mathbf{w} \leftarrow \mathbf{w} + \alpha^{\mathbf{w}}\, \delta\, \nabla \hat{v}(S, \mathbf{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}}\, I\, \delta\, \nabla \ln \pi(A \mid S, \boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$
  end while
end for

This is the **Advantage Actor-Critic (A2C)** algorithm in its one-step, online, episodic form. It is fully **online and incremental**, the policy-gradient analog of TD, Sarsa, Q-learning.

78 Policy Gradient for Continuing Problems

For **continuing problems** (no episode boundaries), performance is defined as the **average reward rate**:

$$\begin{aligned} J(\boldsymbol{\theta}) \doteq r(\pi) &\doteq \lim_{h\to\infty} \frac{1}{h} \sum_{t=1}^{h} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\ &= \lim_{t\to\infty} \mathbb{E}[R_t \mid S_0, A_{0:t-1} \sim \pi] \\ &= \sum_s \mu(s) \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a)\, r, \end{aligned}$$

where $\mu(s) \doteq \lim_{t\to\infty} \Pr\{S_t = s \mid A_{0:t} \sim \pi\}$ is the **steady-state distribution** under π.

Differential return: values are defined as

$$v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s], \qquad q_\pi(s,a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

with

$$G_t \doteq R_{t+1} - r(\pi) + R_{t+2} - r(\pi) + R_{t+3} - r(\pi) + \cdots$$

Continuing Case

With these definitions, the **policy gradient theorem remains true** (now with proportionality constant 1) and **similar algorithms** (REINFORCE, actor-critic) can be used.

79 Policy Parametrization for Continuous Actions

79.1 From Discrete to Continuous Action Spaces

Policy gradient methods can handle **large** and even **continuous action spaces**. Instead of producing a probability for each action, the policy outputs the **statistics of a probability distribution** over actions. A natural choice is the **normal (Gaussian) distribution**:

$$p(x) \doteq \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right).$$

79.2 Gaussian Policy Parametrization

The policy is defined as a normal density over a **real-valued scalar action**:

$$\boxed{\pi(a \mid s, \boldsymbol{\theta}) \doteq \frac{1}{\sigma(s,\boldsymbol{\theta})\sqrt{2\pi}} \exp\left(-\frac{(a-\mu(s,\boldsymbol{\theta}))^2}{2\sigma(s,\boldsymbol{\theta})^2}\right)}$$

where mean and standard deviation are **parametric function approximators** of the state,

$$\mu : \mathcal{S} \times \mathbb{R}^{d'} \to \mathbb{R}, \qquad \sigma : \mathcal{S} \times \mathbb{R}^{d'} \to \mathbb{R}^+.$$

The policy parameters split as $\boldsymbol{\theta} = [\boldsymbol{\theta}_\mu, \boldsymbol{\theta}_\sigma]^\top$.

79.3 Linear Parametrization

The mean can be a **linear** function and the standard deviation the **exponential of a linear** function (must be positive):

$$\mu(s, \boldsymbol{\theta}) \doteq \boldsymbol{\theta}_\mu^\top \mathbf{x}_\mu(s), \qquad \sigma(s, \boldsymbol{\theta}) \doteq \exp(\boldsymbol{\theta}_\sigma^\top \mathbf{x}_\sigma(s)),$$

with $\mathbf{x}_\mu(s)$ and $\mathbf{x}_\sigma(s)$ state feature vectors.

Take-Away

With this Gaussian parametrization, **all the policy gradient algorithms** defined before (REINFORCE, REINFORCE with baseline, actor-critic) can be directly applied to learn to select **real-valued actions**.

References

- R. S. Sutton, A. G. Barto. *Reinforcement Learning: An Introduction.* Second edition. Chapters 9, 10, 13, 16.5.
- R. J. Williams (1992). *Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning.* Machine Learning, 8:229–256. (REINFORCE)
- V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller (2013). *Playing Atari with Deep Reinforcement Learning.* arXiv:1312.5602.
- V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, D. Hassabis (2015). *Human-level Control through Deep Reinforcement Learning.* Nature, 518(7540):529–533.
- V. Mnih, A. Puigdomènech Badia, M. Mirza, A. Graves, T. P. Lillicrap, T. Harley, D. Silver, K. Kavukcuoglu (2016). *Asynchronous Methods for Deep Reinforcement Learning.* arXiv:1602.01783. (A2C, A3C)